

OAEP-NTRU Algorithm Specifications

(PKCKEM-197296-V2)

Shengli Liu You Lyu Shuai Han Bohang Chen
刘胜利 吕优 韩帅 陈博航

Shanghai Jiao Tong University 上海交通大学
{slliu, vergil, dalen17, bohang0918}@sjtu.edu.cn

1	Overview	2
2	Written Specification	4
2.1	Preliminary	4
2.2	Algorithm Specification	5
2.2.1	Polynomial Encoding	9
2.2.2	NTT & InvNTT Implementations	13
2.3	Parameter Sets	15
3	Design Rationale	16
3.1	Hardness Assumptions	16
3.2	ATIF and SOTP	16
3.3	The OAEP [†] Transform	17
3.3.1	OAEP, OAEP+, Q-OAEP: common structure and its dilemma	18
3.3.2	Our OAEP [†] : new structure for flexible pre-image distribution	19
3.4	Decapsulation Failure Probability	21
3.5	Parameter Choices	22
4	Security Analysis	23
4.1	Properties of NTRU-based ATIF	23
4.2	Properties of SOTP	24
4.3	Security of OAEP [†] Transform	25
4.4	Expected Security	26
5	Performance Analysis	28
5.1	Efficiency Analysis of OAEP-NTRU	28
5.2	Performance of OAEP-NTRU Implementation	28
5.3	Performance Comparison with Other KEMs	29

6 Advantages and Limitations	30
6.1 Advantages	30
6.2 Comparison to OAEP, OAEP+ and Q-OAEP	31
6.3 Comparison to FO-based schemes	32
6.4 Limitations	33
A Security Proofs for OAEP[†]	38
A.1 Lemmas for Quantum Random Oracles	38
A.2 Proof of IND-CCA Security	40
A.3 Proofs of Additional Properties	47

1 Overview

OAEP-NTRU is a post-quantum Key Encapsulation Mechanism (KEM) based on a novel transform called OAEP[†], a new post-quantum variant of the OAEP transform, and instantiated from the NTRU lattice problems. It is designed to be a strongly secure and highly efficient KEM, achieving strong security like ANO-CCA (anonymity), PR-CCA (ciphertext pseudo-randomness) and ROB-CCA (robustness) beyond the standard IND-CCA security in the quantum random oracle model (QROM), and supporting an efficient partial message recovery mode where the encapsulation algorithm can also encrypt a short message.

Below we highlight the key features of OAEP-NTRU and OAEP[†] transform.

Novel OAEP[†] Transform beyond FO. The most notable feature of OAEP-NTRU is that it is based on the new transform OAEP[†]. This is in contrast to many other existing KEM schemes, most of which rely on the Fujisaki-Okamoto (FO) transform [23], including the NIST PQC standardized algorithms ML-KEM/Kyber [8] and HQC [24], and the Korean PQC competition finalized algorithm NTRU+ [28].

We note that it is risky to rely on a single paradigm to build cryptographic schemes. Once some weaknesses were found in the paradigm, the security of all schemes based on that paradigm will be affected. It is vital to explore new paradigms in cryptography. From this perspective, the new OAEP[†] enriches the design paradigm for post-quantum KEM, and thus is of significant theoretical and practical importance.

As a new transform, OAEP[†] admits many instantiations. In this document, for better efficiency, we instantiate OAEP[†] from NTRU lattice problems under 128, 256, 512-bit security levels, yielding OAEP-NTRU-648, OAEP-NTRU-1296, OAEP-NTRU-2592, respectively. It is also possible to instantiate OAEP[†] from other lattice problems or isogenies.

Advantages over FO Transform. The new OAEP[†] transform and the NTRU-based instantiation OAEP-NTRU have several advantages over FO:

- (1) **No Re-Encryption & Stronger Security than FO:** OAEP[†] is more side-channel secure than FO, since unlike FO, OAEP[†] does not need re-encryption check in the decapsulation algorithm, which avoids potential

side-channel attacks due to re-encryption and improves the decryption efficiency of OAEP-NTRU. See Subsect. 6.3 for more explanations.

Moreover, $\text{OAEP}^\dagger$ achieves stronger security including ANO-CCA, PR-CCA and ROB-CCA simultaneously, while FO can hardly achieve ROB-CCA (robustness) as noted in [40] (cf. Table 7 in Subsect. 6.3).

- (2) **Comparable Efficiency to FO & New Mode of Partial Message Recovery:** OAEP-NTRU has comparable efficiency to the FO-based KEM schemes like Kyber [8] and NTRU+ [28]. We refer to Table 5 in Subsect. 5.3 for a detailed comparison.

Moreover, OAEP-NTRU supports a more efficient partial message recovery mode, where the encapsulation algorithm can also encrypt a short message besides encapsulating a key, thus achieving more compact ciphertext. See Subsect. 6.1 for more explanations.

- (3) **Lower Decapsulation Failure Probability:** OAEP-NTRU provides strong correctness guarantee. Our parameter analysis shows that the probability of generating a potentially bad key pair is at most 2^{-159} . For the remaining key pairs, decapsulation is perfectly correct: every honestly generated ciphertext can be decapsulated successfully. This is in contrast to many lattice-based KEMs obtained via the FO transform like Kyber [8] and NTRU+ [28], where the decryption-failure probability is averaged over both the randomness of key generation and the randomness used in encapsulation/encryption. Thus, OAEP-NTRU yields a more robust correctness guarantee.

Advantages over other OAEP variants. OAEP-NTRU and the $\text{OAEP}^\dagger$ transform also achieve stronger security & better efficiency than the original OAEP [10] and its variants, like OAEP+ [35] and Q-OAEP [36].

More precisely, the $\text{OAEP}^\dagger$ transform makes use of two novel building blocks, an Almost Trapdoor Injective Function (ATIF) and a Semi-Generalized One-Time Pad (SOTP). This is different from OAEP and OAEP+, which rely on trapdoor permutations over uniform distributions and are thus hard to instantiate post-quantumly, and is also unlike Q-OAEP which requires perfect injectivity of its trapdoor function and are thus hard to instantiate efficiently. We refer to Subsect. 6.2 for a detailed comparison to OAEP variants.

- (1) **Stronger Security than other OAEP variants:** $\text{OAEP}^\dagger$ achieves ANO-CCA, PR-CCA and ROB-CCA simultaneously. In particular, $\text{OAEP}^\dagger$ is the first OAEP variant achieving anonymity and robustness in the QROM (cf. Table 6 in Subsect. 6.2).

Moreover, $\text{OAEP}^\dagger$ has a tighter security with security decay only $\epsilon^{1/2}$ from the property of ATIF, while Q-OAEP has security decay $\epsilon^{1/8}$ (and OAEP and OAEP+ can hardly support post-quantum instantiations).

- (2) **Better Efficiency than other OAEP variants:** OAEP-NTRU has better efficiency than other OAEP variants. This is because the new $\text{OAEP}^\dagger$

transform allows the building block ATIF to be over more flexible non-uniform pre-image distributions and have overwhelming injectivity with negligible error, making it particularly suited for lattice-based instantiations from NTRU, which use centered binomial distributions for efficiency.

Roadmap. The rest of this document is organized as follows.

- In Sect. 2: Written Specification, we specify the algorithms of OAEP-NTRU and present parameter sets targeting different bit-security levels.
- In Sect. 3: Design Rationale, we discuss the design rationale behind OAEP-NTRU, including the underlying hardness assumptions and the OAEP[†] transform instantiated in our construction. We also analyze the decapsulation failure probability and describe the criteria used for parameter selection.
- In Sect. 4: Security Analysis, we present the security theorems of OAEP-NTRU. These include the required IND-CCA security, as well as additional properties such as anonymity, ciphertext pseudorandomness, and robustness. The proofs of these theorems are provided in Appendix A.
- In Sect. 5: Performance Analysis, we experimentally evaluate the performance of OAEP-NTRU using both a reference implementation and an optimized implementation based on the AVX2 instruction set, and compare our performance results with implementations of ML-KEM/Kyber [8] and NTRU+ [28].
- In Sect. 6: Advantages and Limitations, we discuss the advantages and limitations of OAEP-NTRU. We also compare the OAEP[†] transform with existing variants of OAEP, and we compare OAEP-NTRU with widely deployed post-quantum KEMs, which are commonly constructed via the FO transform.

2 Written Specification

2.1 Preliminary

Bytes and arrays. Let $\mathcal{B} = \{0, 1, \dots, 255\}$ be the set of all possible bytes and $\mathcal{I} = \{0, 1, \dots, 65535\}$ be the set of all possible 16-bit integers. For an array b in $\mathcal{B}^l$ or $\mathcal{I}^l$ of length l , we use the bracket notation $b[i]$ to represent the i -th entry of b . We further use bracket notation $b[i : r]$ to represent a slice of the array with $r - i + 1$ entries, from the i -th entry to the r -th entry.

Bitwise Operations. For two bytes (or integers of the same bit-length) a and b , we denote $\&$ for bit-wise ‘and’ operation, $|$ for bit-wise ‘or’ operation, and $\oplus$ for bit-wise ‘xor’ operation. We also denote $\ll$ and $\gg$ for arithmetic left shift and arithmetic right shift, respectively.

Vectors and Polynomials. We use bold lower-case letters to represent a vector over $\mathbb{Z}_q$, like $\mathbf{v} = (\mathbf{v}[0], \mathbf{v}[1], \dots, \mathbf{v}[n-1]) \in \mathbb{Z}_q^n$. Here we use the bracket notation $\mathbf{v}[i]$ to represent the i -th entry of $\mathbf{v}$. We denote by $\mathcal{R}$ the polynomial ring $\mathbb{Z}_q[X]/(f(X))$, where $f(X)$ is a n -degree cyclotomic polynomial and q is a prime integer. We use a vector $\mathbf{g} = (\mathbf{g}[0], \mathbf{g}[1], \dots, \mathbf{g}[n-1]) \in \mathbb{Z}_q^n$ to represent a polynomial $\mathbf{g}[0] + \mathbf{g}[1]X + \dots + \mathbf{g}[n-1]X^{n-1} \in \mathcal{R} = \mathbb{Z}_q[X]/(f(X))$, where $\mathbf{g}[i]$ is the i -th coefficient of the polynomial. We let $\text{plen} = \lceil \log_2 q \rceil n/8$ denote the byte-length of a serialized polynomial. More details about the serialization are discussed in Subsect. 2.2.1.

Number Theoretic Transform (NTT). The Number Theoretic Transform (NTT) is an analogue of the discrete Fourier transform (DFT) defined over a finite field. Let $f(X) = \prod_{i=0}^{n/d-1} f_i(X)$ be the irreducible factorization of the cyclotomic polynomial $f(X)$, where each $f_i(X)$ has degree d . The equality of the degrees follows from a standard property of the cyclotomic polynomial. The NTT defines the following isomorphism:

$$\mathcal{R} = \mathbb{Z}_q[X]/(f(X)) \rightarrow \mathbb{Z}_q[X]/(f_0(X)) \times \mathbb{Z}_q[X]/(f_1(X)) \times \dots \times \mathbb{Z}_q[X]/(f_{n/d-1}(X)),$$

where $\times$ is the direct product operation of rings, and for all $i \in \{0, 1, \dots, n/d-1\}$, $\mathbb{Z}_q[X]/(f_i(X))$ is a finite field. We use bold lower-case letters with hat to represent the image of a polynomial under the NTT, like $\hat{\mathbf{g}} = \text{NTT}(\mathbf{g})$. We use $\odot$ to represent the product operation in the direct product ring $\mathbb{Z}_q[X]/(f_0(X)) \times \dots \times \mathbb{Z}_q[X]/(f_{n/d-1}(X))$. We use InvNTT to represent the inverse NTT operation, like $\text{InvNTT}(\hat{\mathbf{g}}) = \mathbf{g}$. We also view $\hat{\mathbf{g}}$ as a vector in $\mathbb{Z}_q^n$, where the $(id+j)$ -th entry is the j -th coefficient of the polynomial in the i -th field of the direct-product ring, for all $i \in \{0, 1, \dots, n/d-1\}$ and $j \in \{0, 1, \dots, d-1\}$.

Centered Binomial Distribution (CBD). Let CBD_1^n be a distribution over $\{-1, 0, 1\}^n$, where each element has independently identical distribution that takes 0 with probability $1/2$, takes 1 with probability $1/4$ and takes -1 with probability $1/4$.

Hash functions. The hash functions in our scheme are instantiated with either a hash function with a fixed output length of 32 bytes (denoted by $\text{HASH}(m)$, where m is the input), or an eXtensible Output Function (XOF) (denoted by $\text{XOF}(m, l)$, where m is the input and l is the output length in bytes).

Collision Entropy. For a distribution ψ , we define its collision entropy as $-\log_2 \Pr[x = x' : x, x' \leftarrow \psi]$.

2.2 Algorithm Specification

We specify the algorithms of OAEP-NTRU as follows.

- **Main algorithms.** Algs. 1 to 3 show OAEP-NTRU’s key generation algorithm $(pk, sk) \leftarrow \text{KeyGen}$, encapsulation algorithm $(ct, k) \leftarrow \text{Encap}(pk)$, and decapsulation algorithm $k'/\perp \leftarrow \text{Decap}(sk, ct)$, respectively.
 - **Sub-algorithms.** Algs. 4 to 10 and Subsubsects. 2.2.1, 2.2.2 show the sub-algorithms used in KeyGen , Encap , Decap . More precisely, Alg. 4: SOTPEncode and Alg. 5: SOTPDecodeCheck specify the encoding and decoding algorithms of the building block SOTP (Semi-Generalized One-Time Pad), Alg. 6: CBD_1 is used to sample centered binomial distribution (CBD), Alg. 7: BytesToBits is used to convert bytes to bits, Alg. 8: pkDigest is used to compress public keys as digests, and Alg. 9: G and Alg. 10: H are the hash functions.
- Besides, in Subsubsect. 2.2.1, we specify the sub-algorithms EncodePoly and DecodePoly in more detail, which are used to convert between polynomials and bytearrays, and in Subsubsect. 2.2.2, we show the sub-algorithms NTT and InvNTT in more detail.

Algorithm 1 KeyGen

Ensure: Public key pk , secret key sk .

```

1: repeat
2:    $f' \leftarrow_{\$} \mathcal{B}^{n/4}$ 
3:    $\mathbf{f}' := \text{CBD}_1(f')$ 
4:    $\mathbf{f} := 3 \cdot \mathbf{f}' + 1$ 
5:    $\hat{\mathbf{f}} := \text{NTT}(\mathbf{f})$ 
6: until  $\mathbf{f}$  is invertible in  $\mathcal{R}$ 
7: repeat
8:    $g' \leftarrow_{\$} \mathcal{B}^{n/4}$ 
9:    $\mathbf{g}' := \text{CBD}_1(g')$ 
10:   $\mathbf{g} := 3 \cdot \mathbf{g}'$ 
11:   $\hat{\mathbf{g}} := \text{NTT}(\mathbf{g})$ 
12: until  $\mathbf{g}$  is invertible in  $\mathcal{R}$ 
13:  $\hat{\mathbf{h}} := \hat{\mathbf{g}} \odot \hat{\mathbf{f}}^{-1}$ 
14:  $pk := \text{EncodePoly}(\hat{\mathbf{h}})$ 
15:  $sk := \text{EncodePoly}(\hat{\mathbf{f}}) \parallel \text{EncodePoly}(\hat{\mathbf{h}}^{-1}) \parallel \text{pkDigest}(pk)$ 
16: return  $(pk, sk)$ 

```

Algorithm 2 Encap

Require: Public key pk .

Ensure: Ciphertext ct , key k .

```
1:  $m := 0^{n/8} \in \mathcal{B}^{n/8}$ 
2:  $e' \leftarrow_{\$} \mathcal{B}^{n/4}$ 
3:  $\mathbf{e} := \text{CBD}_1(e')$ 
4:  $\hat{\mathbf{h}} := \text{DecodePoly}(pk)$ 
5:  $e := \text{EncodePolyShort}(\mathbf{e})$ 
6:  $r := G(e)$ 
7:  $\mathbf{s} := \text{SOTPEncode}(m, r)$ 
8:  $s := \text{EncodePolyShort}(\mathbf{s})$ 
9:  $\hat{\mathbf{s}} := \text{NTT}(\mathbf{s})$ 
10:  $\hat{\mathbf{e}} := \text{NTT}(\mathbf{e})$ 
11:  $\mathbf{c} := \hat{\mathbf{h}} \odot \hat{\mathbf{s}} - \hat{\mathbf{e}}$ 
12:  $(\sigma, k) := H(\text{pkDigest}(pk) \| s \| e)$ 
13:  $ct := \text{EncodePoly}(\mathbf{c}) \| \sigma$ 
14: return  $(ct, k)$ 
```

Algorithm 3 Decap

Require: Secret key sk , ciphertext ct .

Ensure: Key k .

```
1:  $\hat{\mathbf{c}} := \text{DecodePoly}(ct[0 : \text{plen} - 1])$ 
2:  $\hat{\mathbf{f}} := \text{DecodePoly}(sk[0 : \text{plen} - 1])$ 
3:  $\hat{\mathbf{h}}_{\text{inv}} := \text{DecodePoly}(sk[\text{plen} : 2 \cdot \text{plen} - 1])$ 
4:  $\mathbf{e} := \text{InvNTT}(\hat{\mathbf{c}} \odot \hat{\mathbf{f}}) \bmod 3$ 
5:  $\hat{\mathbf{e}} := \text{NTT}(\mathbf{e})$ 
6:  $\hat{\mathbf{s}} := \hat{\mathbf{h}}_{\text{inv}} \odot (\hat{\mathbf{c}} - \hat{\mathbf{e}})$ 
7:  $\mathbf{s} := \text{InvNTT}(\hat{\mathbf{s}})$ 
8: if  $\mathbf{s} \notin \{-1, 0, 1\}^n$  then
9:   return  $\perp$ 
10: end if
11:  $d := sk[2 \cdot \text{plen} : 2 \cdot \text{plen} + \lambda/4 - 1]$ 
12:  $s := \text{EncodePolyShort}(\mathbf{s})$ 
13:  $e := \text{EncodePolyShort}(\mathbf{e})$ 
14:  $(\sigma', k') := H(d \| s \| e)$ 
15: if  $\sigma' \neq \sigma$  or  $\text{SOTPDecodeCheck}(\mathbf{s}, G(e)) = \perp$  then
16:   return  $\perp$ 
17: else
18:   return  $k'$ 
19: end if
```

Algorithm 4 SOTPEncode

Require: message $m \in \mathcal{B}^{n/8}$, mask $r \in \mathcal{B}^{n/4}$.

Ensure: Short polynomial $\mathbf{s} \in \{-1, 0, 1\}^n$.

- 1: $\mathbf{tmp} := (r[0 : n/8 - 1] \oplus m[0 : n/8 - 1]) \parallel r[n/8 : n/4 - 1]$
 - 2: $\mathbf{s} := \text{CBD}_1^n(\mathbf{tmp})$
 - 3: **return** $\mathbf{s}$
-

Algorithm 5 SOTPDecodeCheck

Require: $\mathbf{y} \in \mathbb{Z}^n$ and mask bytes $r \in \mathcal{B}^{n/4}$.

Ensure: Recovered bitstring $m \in \{0, 1\}^n$ and validity bit fail.

- 1: $t := \text{BytesToBits}(r)$
 - 2: **for** $i = 0$ to $n - 1$ **do**
 - 3: $u[i] := \mathbf{y}[i] + t[i + n]$
 - 4: **end for**
 - 5: **if** $\mathbf{u} \notin \{0, 1\}^n$ **then**
 - 6: **return** $\perp$
 - 7: **else**
 - 8: **return** $\top$
 - 9: **end if**
-

Algorithm 6 CBD₁

Require: Randomness $\text{buf} \in \mathcal{B}^{n/4}$.

Ensure: Short polynomial $\mathbf{s} \in \{-1, 0, 1\}^n$.

- 1: $t := \text{BytesToBits}(\text{buf})$
 - 2: **for** $i = 0$ to $n - 1$ **do**
 - 3: $\mathbf{s}[i] := (t[i] \& 1) - (t[i + n] \& 1)$
 - 4: **end for**
 - 5: **return** $\mathbf{s}$
-

Algorithm 7 BytesToBits

Require: Bytearray $\text{buf} \in \mathcal{B}^l$

Ensure: Bitarray $b \in \{0, 1\}^{8l}$.

- 1: **for** $i = 0$ to $l - 1$ **do**
 - 2: $t_0 := \text{buf}[i]$
 - 3: **for** $j = 0$ to 7 **do**
 - 4: $b[8i + j] := t_0 \& 1$
 - 5: $t_0 := t_0 \gg 1$
 - 6: **end for**
 - 7: **end for**
 - 8: **return** b
-

Algorithm 8 pkDigest

Require: Bytearray data.

Ensure: Bytearray buf $\in \mathcal{B}^{\lambda/4}$.

- 1: buf := HASH(0x00||data)
 - 2: return buf
-

Algorithm 9 G

Require: Bytearray data.

Ensure: Bytearray buf $\in \mathcal{B}^{n/4}$.

- 1: buf := XOF(0x01||data, $n/4$)
 - 2: return buf
-

Algorithm 10 H

Require: Bytearray data.

Ensure: Bytearray buf $\in \mathcal{B}^{\lambda/4+|k|}$.

- 1: buf := XOF(0x05||data, $\lambda/4 + |k|$)
 - 2: return buf
-

2.2.1 Polynomial Encoding

To encode polynomials to bytearrays, we need to specify the EncodePoly and DecodePoly procedures. They convert between a polynomial and a byte array of length `plen`, which depends on the selected parameter set. Our three different parameter sets correspond to polynomial coefficients of different bit lengths, resulting in different encoding procedures and different `plen` accross the parameter sets.

Below are the specifications of EncodePoly and DecodePoly procedures for different parameter sets. Alg. 11 and Alg. 12 are EncodePoly and DecodePoly for the scheme OAEP-NTRU-648. Alg. 13 and Alg. 14 are for the scheme OAEP-NTRU-1296 and OAEP-NTRU-2592.

Algorithm 11 EncodePoly₁₃ for OAEP-NTRU-648

Require: Polynomial $s \in \mathbb{Z}_q^n$

Ensure: Bytearray $\text{buf} \in \mathcal{B}^{13n/8}$.

```
1: Prepare array  $t \in \mathcal{I}^8$ 
2: for  $i = 0$  to  $n/8 - 1$  do
3:   for  $j = 0$  to  $7$  do
4:      $t[j] := s[8i + j]$ 
5:     if  $t[j] < 0$  then
6:        $t[j] := t[j] + q$ 
7:     end if
8:   end for
9:    $\text{buf}[13i] := t[0]$ 
10:   $\text{buf}[13i + 1] := (t[0] \gg 8) \mid (t[1] \ll 5)$ 
11:   $\text{buf}[13i + 2] := (t[1] \gg 3)$ 
12:   $\text{buf}[13i + 3] := (t[1] \gg 11) \mid (t[2] \ll 2)$ 
13:   $\text{buf}[13i + 4] := (t[2] \gg 6) \mid (t[3] \ll 7)$ 
14:   $\text{buf}[13i + 5] := (t[3] \gg 1)$ 
15:   $\text{buf}[13i + 6] := (t[3] \gg 9) \mid (t[4] \ll 4)$ 
16:   $\text{buf}[13i + 7] := (t[4] \gg 4)$ 
17:   $\text{buf}[13i + 8] := (t[4] \gg 12) \mid (t[5] \ll 1)$ 
18:   $\text{buf}[13i + 9] := (t[5] \gg 7) \mid (t[6] \ll 6)$ 
19:   $\text{buf}[13i + 10] := (t[6] \gg 2)$ 
20:   $\text{buf}[13i + 11] := (t[6] \gg 10) \mid (t[7] \ll 3)$ 
21:   $\text{buf}[13i + 12] := (t[7] \gg 5)$ 
22: end for
23: return  $\text{buf}$ 
```

Algorithm 12 DecodePoly₁₃ for OAEP-NTRU-648

Require: Bytearray buf $\in \mathcal{B}^{13n/8}$.

Ensure: Polynomial s $\in \mathbb{Z}_q^n$.

```
1: Prepare array  $t \in \mathcal{I}^{13}$ 
2: for  $i = 0$  to  $n/8 - 1$  do
3:   for  $j = 0$  to 12 do
4:      $t[j] := \text{buf}[13i + j]$ 
5:   end for
6:    $s[8i] := t[0] \mid ((t[1] \ \& \ 0x1f) \ll 8)$ 
7:    $s[8i + 1] := (t[1] \gg 5) \mid (t[2] \ll 3) \mid ((t[3] \ \& \ 0x3) \ll 11)$ 
8:    $s[8i + 2] := (t[3] \gg 2) \mid ((t[4] \ \& \ 0x7f) \ll 6)$ 
9:    $s[8i + 3] := (t[4] \gg 7) \mid (t[5] \ll 1) \mid ((t[6] \ \& \ 0xf) \ll 9)$ 
10:   $s[8i + 4] := (t[6] \gg 4) \mid (t[7] \ll 4) \mid ((t[8] \ \& \ 0x1) \ll 12)$ 
11:   $s[8i + 5] := (t[8] \gg 1) \mid ((t[9] \ \& \ 0x3f) \ll 7)$ 
12:   $s[8i + 6] := (t[9] \gg 6) \mid (t[10] \ll 2) \mid ((t[11] \ \& \ 0x7) \ll 10)$ 
13:   $s[8i + 7] := (t[11] \gg 3) \mid (t[12] \ll 5)$ 
14: end for
15: return s
```

Algorithm 13 EncodePoly₁₅ for OAEP-NTRU-1296/2592

Require: Polynomial $s \in \mathbb{Z}_q^n$.

Ensure: Bytearray $\text{buf} \in \mathcal{B}^{15n/8}$.

```
1: Prepare array  $t \in \mathcal{I}^8$ 
2: for  $i = 0$  to  $n/8 - 1$  do
3:   for  $j = 0$  to  $7$  do
4:      $t[j] := s[8i + j]$ 
5:     if  $t[j] < 0$  then
6:        $t[j] := t[j] + q$ 
7:     end if
8:   end for
9:    $\text{buf}[15i] := t[0]$ 
10:   $\text{buf}[15i + 1] := (t[0] \gg 8) \mid (t[1] \ll 7)$ 
11:   $\text{buf}[15i + 2] := (t[1] \gg 1)$ 
12:   $\text{buf}[15i + 3] := (t[1] \gg 9) \mid (t[2] \ll 6)$ 
13:   $\text{buf}[15i + 4] := (t[2] \gg 2)$ 
14:   $\text{buf}[15i + 5] := (t[2] \gg 10) \mid (t[3] \ll 5)$ 
15:   $\text{buf}[15i + 6] := (t[3] \gg 3)$ 
16:   $\text{buf}[15i + 7] := (t[3] \gg 11) \mid (t[4] \ll 4)$ 
17:   $\text{buf}[15i + 8] := (t[4] \gg 4)$ 
18:   $\text{buf}[15i + 9] := (t[4] \gg 12) \mid (t[5] \ll 3)$ 
19:   $\text{buf}[15i + 10] := (t[5] \gg 5)$ 
20:   $\text{buf}[15i + 11] := (t[5] \gg 13) \mid (t[6] \ll 2)$ 
21:   $\text{buf}[15i + 12] := (t[6] \gg 6)$ 
22:   $\text{buf}[15i + 13] := (t[6] \gg 14) \mid (t[7] \ll 1)$ 
23:   $\text{buf}[15i + 14] := (t[7] \gg 7)$ 
24: end for
25: return  $\text{buf}$ 
```

Algorithm 14 DecodePoly₁₅ for OAEP-NTRU-1296/2592

Require: Bytearray buf $\in \mathcal{B}^{15n/8}$.

Ensure: Polynomial $s \in \mathbb{Z}_q^n$.

```
1: Prepare array  $t \in \mathcal{I}^{15}$ 
2: for  $i = 0$  to  $n/8 - 1$  do
3:   for  $j = 0$  to 14 do
4:      $t[j] := \text{buf}[15i + j]$ 
5:   end for
6:    $s[8i] := t[0] \mid ((t[1] \ \& \ 0x7f) \ll 8)$ 
7:    $s[8i + 1] := (t[1] \gg 7) \mid (t[2] \ll 1) \mid ((t[3] \ \& \ 0x3f) \ll 9)$ 
8:    $s[8i + 2] := (t[3] \gg 6) \mid (t[4] \ll 2) \mid ((t[5] \ \& \ 0x1f) \ll 10)$ 
9:    $s[8i + 3] := (t[5] \gg 5) \mid (t[6] \ll 3) \mid ((t[7] \ \& \ 0xf) \ll 11)$ 
10:   $s[8i + 4] := (t[7] \gg 4) \mid (t[8] \ll 4) \mid ((t[9] \ \& \ 0x7) \ll 12)$ 
11:   $s[8i + 5] := (t[9] \gg 3) \mid (t[10] \ll 5) \mid ((t[11] \ \& \ 0x3) \ll 13)$ 
12:   $s[8i + 6] := (t[11] \gg 2) \mid (t[12] \ll 6) \mid ((t[13] \ \& \ 0x1) \ll 14)$ 
13:   $s[8i + 7] := (t[13] \gg 1) \mid (t[14] \ll 7)$ 
14: end for
15: return  $s$ 
```

We also use the encoding of short polynomials (i.e., polynomials with coefficients in $\{-1, 0, 1\}$). The procedure is provided below.

Algorithm 15 EncodePolyShort

Require: Short Polynomial $s \in \{-1, 0, 1\}^n$

Ensure: Bytearray buf $\in \mathcal{B}^{n/8}$.

```
1: for  $i = 0$  to  $n/4 - 1$  do
2:    $\text{buf}[i] := 0$ 
3:   for  $j = 0$  to 3 do
4:      $\text{buf}[i] := \text{buf}[i] + ((s[4i + j] + 1) \ll (2 * j))$ 
5:   end for
6: end for
7: return buf
```

2.2.2 NTT & InvNTT Implementations

We choose the ring $\mathcal{R} = \mathbb{Z}_q/(f(X))$ with $f(X) = X^n - X^{n/2} + 1$. For $n = 2^i 3^j$ where $i, j > 0$, $f(X)$ is the $3n$ -th cyclotomic polynomial. With the strategy established by Lyubashevsky and Seiler in [31], we can construct fast implementation of NTT over such ring. Same strategy is also used in the parameter choice and NTT implementation of NTRU+[28]. Below we state the procedure of the NTT algorithm.

Let ζ_6 be a primitive 6-th root in $\mathbb{Z}_q$. The fast implementation of such NTT consists of a combination of three types of layers, the initial radix-2 layers, the standard radix-3 layers and the standard radix-2 layers. Denote L_3 the number

of standard radix-3 layers and L_2 the number of standard radix-2 layers. The input polynomial $\mathbf{p} \in \mathcal{R}$ to our algorithm goes through one initial Radix-2 Layer first, then L_3 standard radix-3 layers, and finally L_2 standard radix-2 layers. After that, the algorithm will output the decomposed polynomial

$$\hat{\mathbf{p}} \in \mathbb{Z}_q[X]/(f_0(X)) \times \mathbb{Z}_q[X]/(f_1(X)) \times \cdots \times \mathbb{Z}_q[X]/(f_{n/d-1}(X)).$$

The followings are detailed description of the layers.

Initial Radix-2 Layer: This layer computes the isomorphism

$$\mathcal{R} = \mathbb{Z}_q/(X^n - X^{n/2} + 1) \cong \mathbb{Z}_q/(X^{n/2} - \zeta_6) \times \mathbb{Z}_q/(X^{n/2} - \zeta_6^5).$$

Concretely, on input a polynomial $\mathbf{p} \in \mathcal{R}$, the algorithm computes $p_1 := p \bmod (X^n - \zeta_6)$ and $p_2 := p \bmod (X^n - \zeta_6^5)$ and outputs

$$(p_{11}, p_{12}) \in \mathbb{Z}_q/(X^n - \zeta_6) \times \mathbb{Z}_q/(X^n - \zeta_6^5).$$

Standard Radix-3 Layer: The i -th such layer will have a tuple of $2 \cdot 3^{i-1}$ polynomials as input (which is the output of the previous layer), in which the j -th polynomial p_i for $j \in \{0, 1, \dots, 2 \cdot 3^{i-1} - 1\}$ is in a quotient ring of the form $\mathbb{Z}_q/(x^{3^m} - \alpha_{ij}^3)$, where $\alpha_{ij} \in \mathbb{Z}_q$ and $m = \frac{n}{2 \cdot 3^i}$.

For the j -th polynomial, the layer computes the isomorphism

$$\mathbb{Z}_q/(x^{3^m} - \alpha_{ij}^3) \cong \prod_{k=0}^{2} \mathbb{Z}_q/(X^m - \alpha_{ij} \zeta_6^{2k}).$$

Concretely, for each polynomial p_{ij} , it computes

$$p_{(i+1)(3j+k)} := p_{ij} \bmod (X^m - \alpha_{ij} \zeta_6^{2k}),$$

for $k \in \{0, 1, 2\}$ and outputs the tuple $(p_{(i+1)0}, p_{(i+1)1}, \dots, p_{(i+1)(2 \cdot 3^i - 1)})$.

Standard Radix-2 Layer: The i -th such layer will have a tuple of $2^i \cdot 3^{L_3}$ polynomials, as input (which is the output of the previous layer), in which the j -th polynomial p_i for $j \in \{0, 1, \dots, 2^i \cdot 3^{L_3} - 1\}$ is in a quotient ring of the form $\mathbb{Z}_q/(x^{2^m} - \alpha_{ij}^2)$, where $\alpha_{ij} \in \mathbb{Z}_q$ and $m = \frac{n}{2^{i+1} \cdot 3^{L_3}}$.

For the j -th polynomial, the layer computes the isomorphism

$$\mathbb{Z}_q/(x^{2^m} - \alpha_{ij}^2) \cong \mathbb{Z}_q/(X^m - \alpha_{ij}) \times \mathbb{Z}_q/(X^m - \alpha_{ij} \zeta_6^3)$$

Concretely, for each polynomial p_{ij} , it computes

$$p_{(i+1)(2j+k)} := p_{ij} \bmod (X^m - \alpha_{ij} \zeta_6^{3k}),$$

for $k \in \{0, 1\}$ and outputs the tuple $(p_{(i+1)0}, p_{(i+1)1}, \dots, p_{(i+1)(2^{i+1} \cdot 3^{L_3} - 1)})$

The output of the last standard radix-2 layer is the final output of the NTT algorithm.

As for InvNTT , it is a full inversion of the NTT algorithm. It performs the inverse of each standard radix-2 layers first, then the inverse of each standard radix-3 layers, and finally the inverse of the initial radix-2 layer.

With the butterfly paradigm of NTT and following the analysis in [31], we can implement each layer of NTT and InvNTT in time linear to n , thus obtain an implementation of both algorithm in time $O(n \log n)$ for constant d .

For each parameter set of OAEP-NTRU , the concrete layer configuration (d, L_3, L_2) for NTT is listed in Table 1 in Subsect. 2.3.

2.3 Parameter Sets

We state our choice of the algebraic structure and the parameter sets corresponding to all security levels. The polynomials in our scheme are elements of the polynomial ring $\mathcal{R} = \mathbb{Z}_q/(f(X))$, where $f(X) = X^n - X^{n/2} + 1$. We will choose parameters such that $f(X)$ can be decomposed to n/d irreducible polynomials of degree d . Besides, we decide the length in bytes $|k|$ of the encapsulated key in our algorithms.

Table 1 shows the parameter choices of $(n, q, |k|)$ for the three required security levels, targeting at least $\lambda = 128/256/512$ -bit classical security levels and 80/128/256-bit quantum security levels, respectively. (The concrete classical and quantum security levels of OAEP-NTRU are presented in Table 5 in Subsect. 5.3.) Besides, Table 1 also shows the layer configuration (d, L_3, L_2) of the NTT implementation discussed in Subsubsect. 2.2.2, whose value is determined by (n, q) . We also show in Table 1 the correctness error of OAEP-NTRU (measured by the decapsulation failure probability) under the parameter configurations, whose computation is referred to Subsect. 3.4.

Table 1: Parameter Sets.

Algorithm Name	n	q	$ k $	d	L_3	L_2	Correctness Error	Security Level (in bits)	
								Classical	Quantum
OAEP-NTRU-648	648	7129	32	3	3	2	2^{-159}	≥ 128	≥ 80
OAEP-NTRU-1296	1296	17497	32	2	4	2	2^{-978}	≥ 256	≥ 128
OAEP-NTRU-2592	2592	28513	64	3	3	4	2^{-652}	≥ 512	≥ 256

3 Design Rationale

3.1 Hardness Assumptions

Our scheme OAEP-NTRU relies on the NTRU assumption and Ring-LWE assumptions. We present the definition of the assumptions as follows.

Given a polynomial ring $\mathcal{R}$ and a distribution ψ over $\mathcal{R}^2$, the $\text{NTRU}_{\mathcal{R},\psi}$ assumption is a widely-used post-quantum assumption, which assumes that $\mathbf{h} := \mathbf{g}(p\mathbf{f} + 1)^{-1} \in \mathcal{R}$ for $(\mathbf{f}, \mathbf{g}) \leftarrow \psi$ is indistinguishable from a random element $\mathbf{u} \leftarrow_{\$} \mathcal{R}$. Here we parse $\psi = (\psi_1, \psi_2)$, so $\mathbf{f} \leftarrow \psi_1$ and $\mathbf{g} \leftarrow \psi_2$. We also require that for any $\mathbf{w} \in \mathcal{R}$, it holds that $\Pr[\mathbf{g} = \mathbf{w} : \mathbf{g} \leftarrow \psi_2] \leq \text{negl}(\lambda)$. This requirement is naturally satisfied by the NTRU assumption.

Definition 1 (The $\text{NTRU}_{\mathcal{R},\psi}$ Assumption [26, 19]). Let q be a positive integer, and p be an integer co-prime to q . Let $\mathcal{R} := \mathbb{Z}_q[X]/(f(X))$ for some polynomial $f(X) \in \mathbb{Z}_q[X]$ and ψ be a distribution over $\mathcal{R}^2$. The advantage function of an adversary $\mathcal{A}$ is defined as $\text{Adv}_{\mathcal{R},\psi}^{\text{NTRU}}(\mathcal{A}) := |\Pr[\mathcal{A}(\mathbf{h}) \Rightarrow 1] - \Pr[\mathcal{A}(\mathbf{u}) \Rightarrow 1]|$, where $(\mathbf{f}, \mathbf{g}) \leftarrow \psi$, $\mathbf{h} := \mathbf{g}(p\mathbf{f} + 1)^{-1}$, and $\mathbf{u} \leftarrow_{\$} \mathcal{R}$. The $\text{NTRU}_{\mathcal{R},\psi}$ assumption holds if $\text{Adv}_{\mathcal{R},\psi}^{\text{NTRU}}(\mathcal{A}) = \text{negl}(\lambda)$ for all quantum polynomial-time algorithm $\mathcal{A}$.

Definition 2 (The Computation $\text{RLWE}_{\mathcal{R},\psi}$ Assumption). Let ψ be a distribution over $\mathcal{R}^2$. The computation Ring-LWE ($\text{RLWE}_{\mathcal{R},\psi}$) assumption states that $\text{Adv}_{\mathcal{R},\psi}^{\text{cRLWE}}(\mathcal{A}) := \Pr[\mathbf{e} = \mathbf{e}' : (\mathbf{s}, \mathbf{e}) \leftarrow \psi, \mathbf{h} \leftarrow_{\$} \mathcal{R}, \mathbf{e}' \leftarrow \mathcal{A}(\mathbf{h}, \mathbf{h}\mathbf{s} + \mathbf{e})] = \text{negl}(\lambda)$ for all quantum polynomial-time algorithm $\mathcal{A}$.

Definition 3 (The Decision $\text{RLWE}_{\mathcal{R},\psi}$ Assumption). Let ψ be a distribution over $\mathcal{R}^2$. The decision Ring-LWE ($\text{RLWE}_{\mathcal{R},\psi}$) assumption states that $\text{Adv}_{\mathcal{R},\psi}^{\text{dRLWE}}(\mathcal{A}) := |\Pr[\mathcal{A}(\mathbf{h}, \mathbf{h}\mathbf{s} + \mathbf{e}) \Rightarrow 1] - \Pr[\mathcal{A}(\mathbf{h}, \mathbf{u}) \Rightarrow 1]| = \text{negl}(\lambda)$ for all quantum polynomial-time algorithm $\mathcal{A}$, where $(\mathbf{s}, \mathbf{e}) \leftarrow \psi$ and $\mathbf{h}, \mathbf{u} \leftarrow_{\$} \mathcal{R}$.

3.2 ATIF and SOTP

We define almost trapdoor injective function (ATIF) family and a primitive called semi-generalized one-time pad (SOTP). These are the building blocks for our OAEP[†] transform. We also introduce our instantiation of ATIF and SOTP.

Definition 4 (ATIF). An almost trapdoor injective function (ATIF) family $\mathcal{F} : \mathcal{X} \rightarrow \mathcal{Y}$ consists of the following three algorithms:

- Gen : It outputs (descriptions of) two algorithms $\mathbf{F}$ and $\mathbf{F}^{-1}$.
- $\mathbf{F}(x)$: It is a deterministic algorithm, which takes as input x and outputs the evaluated value $y := \mathbf{F}(x)$.
- $\mathbf{F}^{-1}(y)$: It is a deterministic algorithm which takes as input y and outputs a pre-image x or a special symbol $\perp$.

In addition, we say an ATIF family $\mathcal{F}$ has η -injectivity error, denoted by η -ATIF, if $\Pr[\exists x \in \mathcal{X}, \mathbf{F}^{-1}(\mathbf{F}(x)) \neq x : (\mathbf{F}, \mathbf{F}^{-1}) \leftarrow \text{Gen}] \leq \eta$.

Gen :	F(s, e) :	F ⁻¹ (c ∈ R) :
(f', g) ← ψ, f := 3f' + 1	If s, e ∉ {-1, 0, 1} ⁿ :	Parse F ⁻¹ = (f, h ⁻¹)
If f or g is not invertible in R:	Return ⊥	e := (cf mod q) mod 3
re-sample f', g	Parse F = h	s := h ⁻¹ (c - e)
h := 3gf ⁻¹	c := hs + e	If s ∉ {-1, 0, 1} ⁿ : Return ⊥
Return (F, F ⁻¹) := (h, (f, h ⁻¹))	Return c	Return (s, e)

Figure 1: Almost trapdoor injective function family $\mathcal{F}_{\text{ntru}} = (\text{Gen}, F, F^{-1})$ from NTRU.

We instantiate our ATIF family $\mathcal{F}_{\text{ntru}}$ from NTRU as shown in Fig. 1. To obtain secure KEM from $\mathcal{F}_{\text{ntru}}$, we require the injectivity error of $\mathcal{F}_{\text{ntru}}$ to be negligible. This requires a careful choice of the parameters, and is discussed in Subsect. 2.3.

Next, we present the primitive called Semi-Generalized One-Time Pad (SOTP), which was introduced in [28]. SOTP can be regarded as a generalized one-time pad. It uses Encode and Decode algorithms, rather than the simple “ $\oplus$ ” over binary strings in one-time pad. More importantly, SOTP requires that different codewords encoded from different plaintexts share the same distribution ψ' , rather than the uniform distribution as asked by the one-time pad.

Definition 5 (Semi-Generalized One-Time Pad [28]). A semi-generalized one-time pad $\text{SOTP} = (\text{Encode}, \text{Decode}, \mathcal{X}, \mathcal{U}, \mathcal{Y}, \psi')$ with message space $\mathcal{X}$, randomness space $\mathcal{U}$, and a code space $\mathcal{Y}$ with corresponding distribution ψ' consists of the following two algorithms and satisfies correctness and ψ' -message hiding.

- **Encode**(x, r): It is a deterministic algorithm which takes as input $(x, r) \in \mathcal{X} \times \mathcal{U}$ and outputs a codeword $y \in \mathcal{Y}$.
- **Decode**(y, r): It is a deterministic algorithm which takes as input $(y, r) \in \mathcal{Y} \times \mathcal{U}$ and outputs $x \in \mathcal{X}$.

A SOTP scheme should satisfy the following properties:

Correctness of SOTP. $\forall (x, r) \in \mathcal{X} \times \mathcal{U}$, we have $\text{Decode}(\text{Encode}(x, r), r) = x$.

ψ' -Message Hiding of SOTP. For a fixed $x \in \mathcal{X}$, suppose that the random variable $y \leftarrow \text{Encode}(x, r)$ for $r \leftarrow_s \mathcal{U}$ follows distribution ψ_x over $\mathcal{Y}$. If $\psi_x = \psi'$ for all $x \in \mathcal{X}$, then SOTP has the property of ψ' -message hiding.

We instantiate SOTP as shown in Fig. 2.

3.3 The OAEP[†] Transform

In this subsection, we first recall the common structure of the existing OAEP variants (i.e., OAEP [10], OAEP+ [35] and Q-OAEP [36]) and explain why they are not friendly to post-quantum instantiations. Then we introduce our OAEP[†] transform and present its construction

Encode($\mathbf{m}, \mathbf{r}$) :	Decode($\mathbf{y}, \mathbf{r}$) :
Parse $\mathbf{r} = (\mathbf{r}_1, \mathbf{r}_2) \in \{0, 1\}^n \times \{0, 1\}^n$	Parse $\mathbf{r} = (\mathbf{r}_1, \mathbf{r}_2) \in \{0, 1\}^n \times \{0, 1\}^n$
$\mathbf{y} := (\mathbf{m} \oplus \mathbf{r}_1) - \mathbf{r}_2$	If $(\mathbf{y} + \mathbf{r}_2) \notin \{0, 1\}^n$: Return $\perp$
Return $\mathbf{y}$	$\mathbf{m} := (\mathbf{y} + \mathbf{r}_2) \oplus \mathbf{r}_1$
	Return m

Figure 2: Instantiation for $\text{SOTP} = (\text{Encode}, \text{Decode}, \{0, 1\}^n, \{0, 1\}^{2n}, \mathcal{R}, \text{CBD}_1^n)$, where “ $-$ ” denote the bit-wise subtraction over $\mathbb{Z}$.

3.3.1 OAEP, OAEP+, Q-OAEP: common structure and its dilemma

The OAEP [10] and its variants [35, 36] have a core trapdoor permutation or injective function (F, F^{-1}) and two hash functions G, H' (and additional hash function H'' for Q-OAEP [36]) as building blocks, and they all adopt two-round Feistel network to couple (F, F^{-1}) with G, H' . See Fig. 3 for the common structure of OAEP [10], OAEP+ [35] and Q-OAEP [36] for constructing PKE.¹

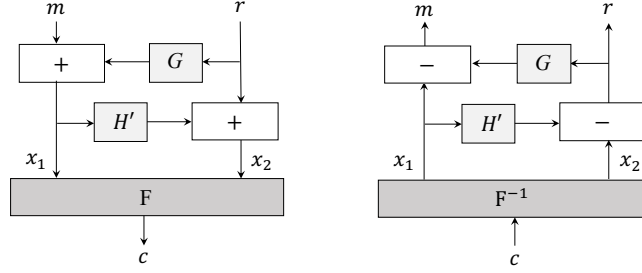


Figure 3: Overview of the common structure of OAEP [10] and its variants [35, 36]: encryption (left) & decryption (right). The ciphertext is $ct := c$.

For example, let m be the plaintext and r be the randomness. Then the ciphertext ct is given by

$$ct := c = F(x_1, x_2) \text{ where } x_1 := G(r) + m \text{ and } x_2 := H'(x_1) + r,$$

and its security heavily relies on the one-wayness of F .

To support decryption, $G(r)$ and $H'(x_1)$ must take some groups $(\mathbb{G}_1, +)$, $(\mathbb{G}_2, +)$ as their co-domains. When G, H' are modeled as random oracles, $G(r)$ and $H'(x_1)$ are uniform over $\mathbb{G}_1, \mathbb{G}_2$. Consequently, OAEP [10] (and its variants [35, 36]) needs one-wayness of $F : \mathcal{X}_1 \times \mathcal{X}_2 \rightarrow \mathcal{Y}$ supported by the uniform distribution over $\mathcal{X}_1 \times \mathcal{X}_2 = \mathbb{G}_1 \times \mathbb{G}_2$.

The two hash functions impose a strict restriction on the distribution of F 's pre-image. This is not very friendly to the current post-quantum trapdoor injective functions. For example, the injective one-way function $\mathbf{y} = F(\mathbf{s}, \mathbf{e}) = \mathbf{A}\mathbf{s} + \mathbf{e}$

¹Fig. 3 is just a rough demonstration, neglecting the padding of m in OAEP [10] and OAEP+ [35], and the additional hash in Q-OAEP [36].

over lattices always requires that $\mathbf{e}$ (even $\mathbf{s}$) satisfy discrete Gaussian, centered binomial, or sparse ternary distributions to accommodate to practical applications. Such distributions are far from uniform distribution and do not work well with OAEP [10]. This partially explains why the existing post-quantum PKE/KEM candidate/final constructions in NIST and Korean PQC competitions all resort to the FO transform [23], rather than OAEP [10], to achieve both efficiency and CCA security.

3.3.2 Our OAEP[†]: new structure for flexible pre-image distribution

To avoid the limitations from the two hash functions, we design new structure for our OAEP[†] to construct PKE. See Fig. 4 for details.

This new transform results in a KEM with partial message recovery [13], where **Encap** is also able to encrypt a message m in the encapsulation ct besides encapsulating a key k . Accordingly, **Decap** can not only recover the key k but also the message m .

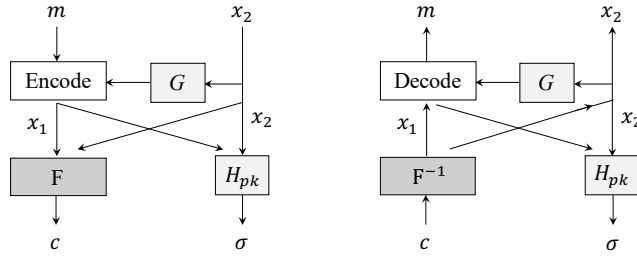


Figure 4: Overview of our OAEP[†]: encryption (left) & decryption (right). The ciphertext is $ct = (c, \sigma)$, where $c := F(x_1, x_2)$ for $x_1 := \text{Encode}(m, G(x_2))$ and $\sigma := H_{pk}(x_1, x_2)$.

Firstly, we remove the hash function $H'(\cdot)$ and let $r = x_2$ directly serve as the second input of F . Therefore, x_2 gets rid of the limitation from H' , and it can freely take any distribution ψ_2 over $\mathcal{X}_2$.

Secondly, we replace the group operation “+” (in encryption) and “−” (in decryption) with the **Encode** and **Decode** algorithms of a so-called semi-generalized one-time pad (SOTP) scheme [28]. Consequently, $x_1 := \text{Encode}(m, G(x_2))$ serves as the first input of F , and $m := \text{Decode}(x_1, G(x_2))$ can be correctly recovered. We require that $\text{Encode}(m, r)$ has a same distribution ψ_1 for any m when r is uniform. This is captured by the ψ_1 -message hiding property of SOTP. If G is a random oracle and $G(x_2)$ is uniform, then $x_1 := \text{Encode}(m, G(x_2))$ follows the distribution ψ_1 . We stress that ψ_1 is not necessarily a uniform distribution.

Now the pre-image distribution is $\psi = (\psi_1, \psi_2)$ for the core function F , so we only require one-wayness of F for $\psi = (\psi_1, \psi_2)$, admitting more flexible choices for the pre-image distribution.

Finally, to support CCA security, there must be a mechanism to reject invalid ciphertexts in decryption. Different from OAEP and OAEP+, our OAEP[†]

does not have padding in m , so we have to explore a new way of rejecting invalid ciphertexts. To this end, we introduce a hash function $H(pk, x_1, x_2) = H_{pk}(x_1, x_2)$ to compute an authenticator, and add it in the ciphertext. Now the ciphertext is $ct = (c, \sigma)$. During decryption, x_1, x_2 can be recovered from $F^{-1}(c)$ and the validity of ct can be checked with $\sigma \stackrel{?}{=} H(pk, x_1, x_2)$.

We now describe the $\text{OAEP}^\dagger$ transformation in detail. Below are the building blocks needed in our $\text{OAEP}^\dagger$ transform.

- An ATIF family $\mathcal{F} : \mathcal{X}_1 \times \mathcal{X}_2 \rightarrow \mathcal{Y}$.
- A SOTP scheme $\text{SOTP} = (\text{Encode}, \text{Decode}, \mathcal{M}, \mathcal{U}, \mathcal{X}_1, \psi_1)$
- Two hash functions $G : \mathcal{X}_2 \rightarrow \mathcal{U}$ and $H : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$.
- Let ψ_2 be a distribution on $\mathcal{X}_2$ with collision entropy κ .

The generic construction of $\text{KEM} = (\text{KeyGen}, \text{Encap}, \text{Decap})$ via our $\text{OAEP}^\dagger$ transform is shown in Fig. 5.

KeyGen :	Encap($pk = F, m$) :	Decap($sk = F^{-1}, ct = (c, \sigma)$)
$(F, F^{-1}) \leftarrow \text{Gen}$	$x_2 \leftarrow \psi_2$	$(x_1, x_2) := F^{-1}(c)$
$pk := F$	$x_1 := \text{Encode}(m, G(x_2))$	If $(x_1, x_2) = \perp$: Return $\perp$
$sk := F^{-1}$	$c := F(x_1, x_2)$	$(\sigma', k) := H(pk, x_1, x_2)$
Return (pk, sk)	$(\sigma, k) := H(pk, x_1, x_2)$	If $\sigma' \neq \sigma$: Return $\perp$
	Return $(ct = (c, \sigma), k)$	$m := \text{Decode}(x_1, G(x_2))$
		Return (m, k)

Figure 5: Scheme KEM with partial message recovery constructed from ATIF family $\mathcal{F}$ and SOTP via $\text{OAEP}^\dagger$.

As we have discussed, this transformation turns an ATIF family $\mathcal{F}$ and a SOTP scheme into a CCA-secure KEM supporting partial message recovery. To obtain an ordinary KEM scheme, we modify **Encap** to sample uniform $m \leftarrow_s \mathcal{M}$ instead of requiring m to be an input and modify **Decap** accordingly. This gives us the scheme shown in Fig. 6.

KeyGen :	Encap($pk = F$) :	Decap($sk = F^{-1}, ct = (c, \sigma)$)
$(F, F^{-1}) \leftarrow \text{Gen}$	$x_2 \leftarrow \psi_2$	$(x_1, x_2) := F^{-1}(c)$
$pk := F$	$m \leftarrow_s \mathcal{M}$	If $(x_1, x_2) = \perp$: Return $\perp$
$sk := F^{-1}$	$x_1 := \text{Encode}(m, G(x_2))$	$(\sigma', k) := H(pk, x_1, x_2)$
Return (pk, sk)	$c := F(x_1, x_2)$	If $\sigma' \neq \sigma$: Return $\perp$
	$(\sigma, k) := H(pk, x_1, x_2)$	If $\text{Decode}(x_1, G(x_2)) = \perp$:
	Return $(ct = (c, \sigma), k)$	Return $\perp$
		Return k

Figure 6: Scheme $\text{KEM}_{\text{plain}}$ constructed from ATIF family $\mathcal{F}$ and SOTP via $\text{OAEP}^\dagger$.

In our scheme, we use the $\mathcal{F}_{\text{ntu}}$ family shown in Fig. 1, the SOTP scheme $\text{SOTP} = (\text{Encode}, \text{Decode}, \{0, 1\}^n, \{0, 1\}^{2n}, \mathcal{R}, \text{CBD}_1^n)$ shown in Fig. 2, and the distribution $\psi_2 = \text{CBD}_1^n$ as building blocks to obtain a KEM with the OAEP[†] transform.

3.4 Decapsulation Failure Probability

We say that KEM has a (worst-case) correctness error δ if

$$\mathbb{E}_{pk, sk} \left[\Pr \left[\begin{array}{l} (c, K) \leftarrow \text{Encap}(pk) \\ K' \leftarrow \text{Decap}(sk, c) \end{array} : K \neq K' \right] \right] \leq \delta,$$

where the expectation is taken over $(pk, sk) \leftarrow \text{KeyGen}$ and the choice of the random oracles involved (if any).

Suppose the underlying ATIF has η -injectivity, we have

$$\begin{aligned} & \mathbb{E}_{pk, sk} \left[\Pr \left[\begin{array}{l} (c, K) \leftarrow \text{Encap}(pk) \\ K' \leftarrow \text{Decap}(sk, c) \end{array} : K \neq K' \right] \right] \\ & \leq \mathbb{E}_{pk, sk} \left[\Pr \left[\exists x_1, x_2, F^{-1}(F(x_1, x_2)) \neq (x_1, x_2) \right] \right] \\ & \leq \Pr_{pk, sk} \left[\exists x_1, x_2, F^{-1}(F(x_1, x_2)) \neq (x_1, x_2) \right] \leq \eta. \end{aligned}$$

Recall the algorithm F and F^{-1} in Fig. 1. For any $\mathbf{s} \in \{-1, 0, 1\}^n$, $\mathbf{e} \in \{-1, 0, 1\}^n$ and $\mathbf{c} := \mathbf{h}\mathbf{s} + \mathbf{e}$, if F^{-1} can recover $\mathbf{e}$ correctly, then $\mathbf{s}$ can also be correctly recovered with $\mathbf{s} := \mathbf{h}^{-1}(\mathbf{c} - \mathbf{e})$.

Therefore, for any $\mathbf{s} \in \{-1, 0, 1\}^n$, $\mathbf{e} \in \{-1, 0, 1\}^n$ and $\mathbf{c} := \mathbf{h}\mathbf{s} + \mathbf{e}$, if $(\mathbf{g}, \mathbf{f}') \leftarrow \text{CBD}_1^n$, we have

$$\begin{aligned} \Pr [F^{-1}(F(\mathbf{s}, \mathbf{e})) \neq (\mathbf{s}, \mathbf{e})] & \leq \Pr [(3(\mathbf{g}\mathbf{s} + \mathbf{f}'\mathbf{e}) + \mathbf{e} \bmod q) \bmod 3 \neq \mathbf{e}] \\ & \leq \Pr [|\mathbf{e}|_\infty > (q-1)/2] \leq \Pr [|\mathbf{g}\mathbf{s} + \mathbf{f}'\mathbf{e}|_\infty \geq (q-1)/6]. \end{aligned}$$

Recall that the polynomial multiplication is over $\mathcal{R} = \mathbb{Z}_q/(X^n - X^{n/2} + 1)$. Following the analysis in Subsect. 3.2 in [31], the degree- i coefficient of $\mathbf{g}\mathbf{s} + \mathbf{f}'\mathbf{e}$ is the sum of (at worst) n independent random variables $ba + b'(a + a')$, where b and b' are fixed number in $\{-1, 0, 1\}$ from $(\mathbf{s}, \mathbf{e})$, and $a, a' \leftarrow \text{CBD}_1$ from $(\mathbf{g}, \mathbf{f}')$. Since the injectivity error considers the worst case, we consider the degree- i coefficient of $\mathbf{g}\mathbf{s} + \mathbf{f}'\mathbf{e}$ as the sum of n independent random variables $c = |a| + |a + a'|$, where $a, a' \leftarrow \text{CBD}_1$. The random variable c equals to $\{0, 1, 2, 3\}$ with probability $\{2/8, 3/8, 2/8, 1/8\}$, respectively. Let $\rho(i, j)$ be the probability that the sum of i independent random variables is j . Then we have

$$\rho(i, j) = \frac{2}{8}\rho(i-1, j) + \frac{3}{8}\rho(i-1, j-1) + \frac{2}{8}\rho(i-1, j-2) + \frac{1}{8}\rho(i-1, j-3).$$

With initial values $\rho(1, 0), \rho(1, 1), \rho(1, 2)$, and $\rho(1, 3)$, we can compute $\rho(n, j)$ efficiently. Finally, applying union bound to all n coefficients, we have

$$\eta = \Pr [F^{-1}(F(\mathbf{s}, \mathbf{e})) \neq (\mathbf{s}, \mathbf{e}) : (\mathbf{F}, F^{-1}) \leftarrow \text{Gen}] \leq n \cdot \sum_{j=(q-1)/6}^{3n} \rho(n, j).$$

The concrete parameters and its correctness error are shown in Table 1.

3.5 Parameter Choices

Our parameters are chosen to ensure the security, correctness and efficiency of OAEP-NTRU simultaneously. More precisely, the security of OAEP-NTRU requires the hardness of the underlying NTRU and RLWE assumptions and the collision entropy of $\psi = (\text{CBD}_1^n, \text{CBD}_1^n)$; the correctness of OAEP-NTRU depends on the injectivity error of $\mathcal{F}_{\text{ntru}}$; the efficiency of OAEP-NTRU needs an NTT-friendly implementation. Below we elaborate these considerations:

The concrete hardness of NTRU and RLWE assumptions. We rely on the hardness of NTRU and RLWE assumptions. The concrete hardness of NTRU and RLWE under our choice of parameters, with respect to the given distribution $\psi_1 = \psi_2 = \text{CBD}_1^n$, must satisfy the requirement of the given security parameter. We refer to Subsect. 4.4 for more details.

Collision entropy of the distribution $\psi = (\text{CBD}_1^n, \text{CBD}_1^n)$. We require the collision entropy of the distribution to be larger than 4λ to ensure that the adversary is unable to exploit the collision of this distribution to break security of our KEM scheme. This bound corresponds to the requirement in our security proof of OAEP[†].

For any two random variable $\mathbf{a}, \mathbf{b} \leftarrow \text{CBD}_1^n$, it holds that $\Pr[\mathbf{a} = \mathbf{b}] \leq 2^{-n}$. So the collision entropy of CBD_1^n is at least n bits. Therefore, we require that $n > 4\lambda$.

Injectivity error of $\mathcal{F}_{\text{ntru}}$. The security proof of our scheme relies on an injective F output by Gen in $\mathcal{F}_{\text{ntru}}$. To rule out the case of non-injective F, we require that $\mathcal{F}_{\text{ntru}}$ is instantiated such that the injectivity error η is negligible. This also affects the correctness error of the resulting KEM. We have discussed this in detail in Subsect. 3.4.

NTT-friendliness. To utilize NTT for better efficiency, we need to pick an NTT-friendly ring $\mathcal{R}$. By NTT-friendly, we mean that the NTT isomorphism

$$\mathcal{R} \rightarrow \mathbb{Z}_q[X]/(f_0(X)) \times \mathbb{Z}_q[X]/(f_1(X)) \times \cdots \times \mathbb{Z}_q[X]/(f_{n/d-1}(X)),$$

must satisfy that (1) it is efficient to compute; (2) the degree of each irreducible polynomial $f_i(X)$ for $i \in \{0, 1, \dots, n/d - 1\}$ is small. Furthermore, to obtain a scheme that exactly meets a desired security level, it is desirable to have sufficient flexibility in the choice of the polynomial degree.

Therefore, we follow the technique from [31] and use $\mathcal{R} = \mathbb{Z}_q[X]/(X^n - X^{n/2} + 1)$, where $n = 2^i 3^j$ and $q \equiv 1 \pmod{3}$. An appropriate choice of q and n will allow decomposing the ring into direct product of quotient fields of irreducible polynomials with degree 2 or 3, using only Radix-2 and Radix-3

NTT layers. This optimizes both the cost of operations in the NTT domain and the cost of invoking the NTT transform and its inverse.

4 Security Analysis

Our OAEP-NTRU achieves the required IND-CCA security. In addition, it enjoys anonymity and robustness under chosen-ciphertext attacks. The security is invested by first providing a reduction from the hardness assumptions $\text{NTRU}_{\mathcal{R},\psi}$ and $\text{RLWE}_{\mathcal{R},\psi}$ to breaking the security notions of KEM, and then discussing the concrete expected security of the two underlying assumptions given the corresponding distribution $\psi = \text{CBD}_1^n$.

The theoretical security analysis of our scheme is in two folds. First, we introduce some crucial properties of ATIF and SOTP, and show that our instantiation satisfies these properties assuming the NTRU assumption and the RLWE assumption. Then, we show the CCA security of the KEM obtained from $\text{OAEP}^\dagger$ transform assuming the properties of the ATIF and SOTP primitives. The proof is provided in the Quantum Random Oracle Model (QROM).

4.1 Properties of NTRU-based ATIF

We require the following properties for the ATIF family to obtain CCA security of the KEM obtained from $\text{OAEP}^\dagger$ transform.

Definition 6 (δ correctness error). We say an almost injective function family has δ correctness error if the following holds:

$$\Pr[(F, F^{-1}) \leftarrow \text{Gen}, (x_1, x_2) \leftarrow_{\$} \mathcal{X}_1 \times \mathcal{X}_2 : F^{-1}(F(x_1, x_2)) \neq (x_1, x_2)] \leq \delta.$$

Definition 7 (Partial injectivity). We say an almost injective function family has partial injectivity if for all $(F, F^{-1}) \leftarrow \text{Gen}$, it holds that

$$\max_{x_1, c} |\{x_2 : F(x_1, x_2) = c\}| = 1.$$

Definition 8 (Rigidity of ATIF [12]). An ATIF family $\mathcal{F}$ has rigidity if for all $(F, F^{-1}) \leftarrow \text{Gen}$ and all $y \in \mathcal{Y}$ with $x := F^{-1}(y) \neq \perp$, it holds that $F(x) = y$.

Definition 9 (Partial Domain One-Wayness of ATIF w.r.t. Distribution ψ). Let $\mathcal{F} : \mathcal{X}_1 \times \mathcal{X}_2 \rightarrow \mathcal{Y}$ be an ATIF family, and $\psi = (\psi_1, \psi_2)$ be a distribution over $\mathcal{X}_1 \times \mathcal{X}_2$. The advantage function of an adversary $\mathcal{A}$ against partial domain one-wayness (PDOW) of $\mathcal{F}$ w.r.t. distribution ψ is defined by

$$\text{Adv}_{\mathcal{F}, \psi}^{\text{PD-OW}}(\mathcal{A}) := \Pr \left[x'_2 = x_2^* : \begin{array}{l} (F, F^{-1}) \leftarrow \text{Gen}, (x_1^*, x_2^*) \leftarrow \psi \\ c^* := F(x_1^*, x_2^*), x'_2 \leftarrow \mathcal{A}(F, c^*) \end{array} \right].$$

Similarly, we define $\text{Adv}_{F, F^{-1}, \psi}^{\text{PD-OW}}(\mathcal{A})$ as the advantage of $\mathcal{A}$ against partial domain one-wayness for a fixed pair (F, F^{-1}) . Then $\text{Adv}_{\mathcal{F}, \psi}^{\text{PD-OW}}(\mathcal{A}) = \mathbb{E}_{F, F^{-1}} \left[\text{Adv}_{F, F^{-1}, \psi}^{\text{PD-OW}}(\mathcal{A}) \right]$.

Definition 10 (Pseudo-Randomness of ATIF w.r.t. Distribution ψ). Let $\mathcal{F} : \mathcal{X}_1 \times \mathcal{X}_2 \rightarrow \mathcal{Y}$ be an almost injective function (ATIF) family, and ψ be a distribution over $\mathcal{X}_1 \times \mathcal{X}_2$. The advantage function of an adversary $\mathcal{A}$ against pseudo-randomness of $\mathcal{F}$ w.r.t. distribution ψ is defined by

$$\text{Adv}_{\mathcal{F}, \psi}^{\text{PR}}(\mathcal{A}) := \left| \Pr[b = b' : b' \leftarrow \mathcal{A}(F, c_b)] - \frac{1}{2} \right|,$$

where $(F, F^{-1}) \leftarrow \text{Gen}$, $b \leftarrow_{\$} \{0, 1\}$, $(x_1^*, x_2^*) \leftarrow \psi$, $c_0 := F(x_1^*, x_2^*)$, and $c_1 \leftarrow_{\$} \mathcal{Y}$.

Definition 11 (Collision Resistance of ATIF). Let $\mathcal{F}$ be an almost injective function (ATIF) family. $\mathcal{F}$ has collision resistance if

$$\Pr[F_0 = F_1 : (F_0, F_0^{-1}) \leftarrow \text{Gen}, (F_1, F_1^{-1}) \leftarrow \text{Gen}] = \text{negl}(\lambda).$$

Now, we show the properties of the ATIF family $\mathcal{F}_{\text{ntru}}$ we use in our construction of KEM.

Theorem 12. *The ATIF family $\mathcal{F}_{\text{ntru}} : \{-1, 0, 1\}^n \times \{-1, 0, 1\}^n \rightarrow \mathcal{R}$ in Fig. 1 has partial one-wayness, pseudo-randomness, collision resistance, and rigidity, if the $\text{NTRU}_{\mathcal{R}, \psi}$ assumption, the computation $\text{RLWE}_{\mathcal{R}, \psi}$, and the decision $\text{RLWE}_{\mathcal{R}, \psi}$ hold.*

Proof. We prove these properties for $\mathcal{F}_{\text{ntru}}$ as follows.

Partial One-Wayness w.r.t. ψ . We show this property follows from the $\text{NTRU}_{\mathcal{R}, \psi}$ and $\text{RLWE}_{\mathcal{R}, \psi}$ assumptions.

First, we change the generation of $F = \mathbf{h}$ from $\mathbf{h} := \mathbf{g}(p\mathbf{f}' + 1)^{-1}$ to $\mathbf{h} \leftarrow_{\$} \mathcal{R}$. This change is indistinguishable due to the NTRU assumption. Next, the computational RLWE assumption implies the hardness of recovering $\mathbf{e}$ from $\mathbf{h}$ and $\mathbf{c} := \mathbf{h}\mathbf{s} + \mathbf{e}$. Therefore, $\text{Adv}_{\mathcal{F}_{\text{ntru}}, \psi}^{\text{PD-OW}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{R}, \psi}^{\text{NTRU}}(\mathcal{A}_1) + \text{Adv}_{\mathcal{R}, \psi}^{\text{RLWE}}(\mathcal{A}_2)$ for any quantum algorithm $\mathcal{A}_1$ and $\mathcal{A}_2$. So the partial one-wayness of $\mathcal{F}_{\text{ntru}}$ follows from the NTRU and RLWE assumptions.

Pseudo-Randomness w.r.t. ψ & Collision Resistance. The pseudo-randomness of $\mathcal{F}_{\text{ntru}}$ also follows the NTRU and RLWE assumptions. We first make an indistinguishable change of $\mathbf{h}$ from $\mathbf{h} := \mathbf{g}(p\mathbf{f}' + 1)^{-1}$ to $\mathbf{h} \leftarrow_{\$} \mathcal{R}$, where the indistinguishability is justified by the NTRU assumption. Next, the decision RLWE assumption implies the pseudo-randomness of $\mathcal{F}_{\text{ntru}}$, so $\text{Adv}_{\mathcal{F}_{\text{ntru}}, \psi}^{\text{PR}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{R}, \psi}^{\text{NTRU}}(\mathcal{A}_1) + \text{Adv}_{\mathcal{R}, \psi}^{\text{RLWE}}(\mathcal{A}_2)$.

$\mathcal{F}_{\text{ntru}}$ has collision resistance since $\Pr[\mathbf{h}_1 = \mathbf{h}_2] = \Pr[3\mathbf{g}_1\mathbf{f}_1^{-1} = 3\mathbf{g}_2\mathbf{f}_2^{-1}] = \Pr[\mathbf{g}_1 = \mathbf{g}_2\mathbf{f}_2^{-1}\mathbf{f}_1 : \mathbf{g}_1 \leftarrow \psi_2] = \text{negl}(\lambda)$.

Rigidity. For any $\mathbf{c} \in \mathcal{R}$, if $(\mathbf{s}, \mathbf{e}) := F^{-1}(c) \neq \perp$, then we have $\mathbf{s} = \mathbf{h}^{-1}(\mathbf{c} - \mathbf{e})$. So $\mathbf{c} = \mathbf{h}\mathbf{s} + \mathbf{e} = F(\mathbf{s}, \mathbf{e})$ always holds. $\square$

4.2 Properties of SOTP

We show that the SOTP scheme $\text{SOTP} = (\text{Encode}, \text{Decode}, \{0, 1\}^n, \{0, 1\}^{2n}, \mathcal{R}, \text{CBD}_1^n)$ satisfy the required properties, i.e., correctness and CBD_1^n -message hiding, as defined in Subsect. 3.2.

Theorem 13. *The scheme $\text{SOTP} = (\text{Encode}, \text{Decode}, \{0, 1\}^n, \{0, 1\}^{2n}, \mathcal{R}, \text{CBD}_1^n)$ in Fig. 2 has correctness and CBD_1^n -Message hiding of SOTP.*

Proof. We prove these properties of SOTP as follows.

Correctness of SOTP. For all $\mathbf{m} \in \{0, 1\}^n$ and $\mathbf{r} \in \{0, 1\}^{2n}$, we have

$$\text{Decode}(\text{Encode}(\mathbf{m}, \mathbf{r}), \mathbf{r}) = ((\mathbf{m} \oplus \mathbf{r}_1) - \mathbf{r}_2 + \mathbf{r}_2) \oplus \mathbf{r}_1 = \mathbf{m}.$$

CBD_1^n -Message hiding of SOTP. For any $\mathbf{m} \in \{0, 1\}^n$, if $\mathbf{r}_1, \mathbf{r}_2 \leftarrow_{\$} \{0, 1\}^n$, then $\mathbf{m} \oplus \mathbf{r}_1$ is uniform over $\{0, 1\}^n$ and $\mathbf{y} := (\mathbf{m} \oplus \mathbf{r}_1) - \mathbf{r}_2$ follows the distribution of CBD_1^n . $\square$

4.3 Security of $\text{OAEP}^\dagger$ Transform

We prove the security of the $\text{OAEP}^\dagger$ transform that obtains a KEM with partial message recovery. This implies the security of the version yielding an ordinary KEM scheme.

In this section, we first provide the security definition of a KEM with partial message recovery. Then, we discuss the security of our KEM with partial message recovery from ATIF and SOTP via $\text{OAEP}^\dagger$, showing that our scheme is secure when the required properties of ATIF and SOTP are satisfied.

Below we define for KEM its indistinguishable security, anonymity, and robustness under chosen-ciphertext attacks. These security notions are denoted by IND-CCA, ANO-CCA, and ROB-CCA, respectively. In addition, we also define a security notion PR-CCA for its pseudo-randomness that is stronger than ANO-CCA (i.e., PR-CCA implies ANO-CCA).

Definition 14 (Security Notions for KEM). For any adversary $\mathcal{A}$ and attack $\text{ATK} \in \{\text{IND}, \text{PR}, \text{ANO}\}$ and ROB-CCA, the advantage function against ATK-CCA security for KEM is defined as

$$\text{Adv}_{\text{KEM}}^{\text{ATK-CCA}}(\mathcal{A}) := \left| \Pr \left[\text{Exp}_{\text{KEM}}^{\text{ATK-CCA}} \Rightarrow 1 \right] - \frac{1}{2} \right|, \quad \text{Adv}_{\text{KEM}}^{\text{ROB-CCA}}(\mathcal{A}) := \Pr \left[\text{Exp}_{\text{KEM}}^{\text{ROB-CCA}} \Rightarrow 1 \right],$$

where $\text{Exp}_{\text{KEM}}^{\text{IND-CCA}}$, $\text{Exp}_{\text{KEM}}^{\text{PR-CCA}}$, $\text{Exp}_{\text{KEM}}^{\text{ANO-CCA}}$, and $\text{Exp}_{\text{KEM}}^{\text{ROB-CCA}}$ are given in Fig. 7.

With these security notions, we can state the theorem of the security of our KEM scheme (with partial message recovery) obtained from the $\text{OAEP}^\dagger$ as follows.

Theorem 15 (IND-CCA Security of $\text{OAEP}^\dagger$). *Suppose that $\mathcal{F}$ is an η -ATIF family with rigidity and partial domain one-wayness w.r.t. $\psi = (\psi_1, \psi_2)$, $\text{SOTP} = (\text{Encode}, \text{Decode}, \mathcal{M}, \mathcal{U}, \mathcal{X}_1, \psi_1)$ is a semi-generalized one-time pad scheme, and ψ_2 is a distribution with collision entropy κ . Then scheme KEM from $\text{OAEP}^\dagger$ in Fig. 5 has IND-CCA security. More precisely, for any adversary $\mathcal{A}$ issuing q_G and q_H (quantum) random oracle queries to G and H , and q_d decryption queries, if $q = q_G + q_H + q_d + 2$, then we have*

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq 16q^2 \sqrt{\text{Adv}_{\mathcal{F}, \psi}^{\text{PD-OW}}(\mathcal{B})} + 4q \sqrt{\frac{300q^3}{2^\lambda} + \frac{16\sqrt{2}q^2}{2^{\lambda/2}} + \frac{6\sqrt{q}}{2^{\kappa/4}}} + \eta.$$

$\text{Exp}_{\text{KEM}}^{\text{IND-CCA}}:$ $b \leftarrow_{\mathcal{S}} \{0, 1\}$ $(pk, sk) \leftarrow \text{KeyGen}$ $(m_0, m_1, st) \leftarrow \mathcal{A}^{\text{DECAP}_{\perp}(\cdot)}(pk)$ $(ct^*, k^*) \leftarrow \text{Encap}(pk, m_b)$ $k_0^* := k^*, k_1^* \leftarrow_{\mathcal{S}} \mathcal{K}$ $b' \leftarrow \mathcal{A}^{\text{DECAP}_{ct^*}(sk, \cdot)}(st, ct^*, k_b^*)$ Return $b = b'$	$\text{Exp}_{\text{KEM}}^{\text{PR-CCA}}:$ $b \leftarrow_{\mathcal{S}} \{0, 1\}$ $(pk, sk) \leftarrow \text{KeyGen}$ $(m, st) \leftarrow \mathcal{A}^{\text{DECAP}_{\perp}(\cdot)}(pk)$ $(ct_0^*, k_0^*) \leftarrow \text{Encap}(pk, m)$ $ct_1^* \leftarrow_{\mathcal{S}} \mathcal{CT}, k_1^* \leftarrow_{\mathcal{S}} \mathcal{K}$ $b' \leftarrow \mathcal{A}^{\text{DECAP}_{ct_b^*}(sk, \cdot)}(st, ct_b^*, k_b^*)$ Return $b = b'$	$\text{DECAP}_{ct^*}(ct):$ If $ct = ct^*$: Return $\perp$ $(m, k) \leftarrow \text{Decap}(sk, ct)$ Return (m, k)
$\text{Exp}_{\text{KEM}}^{\text{ANO-CCA}}:$ $b \leftarrow_{\mathcal{S}} \{0, 1\}$ $(pk_0, sk_0) \leftarrow \text{KeyGen}$ $(pk_1, sk_1) \leftarrow \text{KeyGen}$ $(m, st) \leftarrow \mathcal{A}^{\text{DECAP}'_{\perp}(\cdot, \cdot)}(pk_0, pk_1)$ $(ct^*, k^*) \leftarrow \text{Encap}(pk_b, m)$ $b' \leftarrow \mathcal{A}^{\text{DECAP}'_{ct^*}(\cdot, \cdot)}(st, ct^*, k^*)$ Return $b = b'$	$\text{Exp}_{\text{KEM}}^{\text{ROB-CCA}}:$ $(pk_0, sk_0) \leftarrow \text{KeyGen}$ $(pk_1, sk_1) \leftarrow \text{KeyGen}$ $ct \leftarrow \mathcal{A}^{\text{DECAP}'_{\perp}(\cdot, \cdot)}(pk_0, pk_1)$ $(m_0, k_0) \leftarrow \text{Decap}(sk_0, ct)$ $(m_1, k_1) \leftarrow \text{Decap}(sk_1, ct)$ Return $(m_0, k_0) \neq \perp \wedge (m_1, k_1) \neq \perp$	$\text{DECAP}'_{ct^*}(\beta, ct):$ If $ct = ct^*$: Return $\perp$ If $\beta \notin \{0, 1\}$: Return $\perp$ $(m, k) \leftarrow \text{Decap}(sk_{\beta}, ct)$ Return (m, k)

Figure 7: Experiments $\text{Exp}_{\text{KEM}}^{\text{IND-CCA}}$, $\text{Exp}_{\text{KEM}}^{\text{PR-CCA}}$, $\text{Exp}_{\text{KEM}}^{\text{ANO-CCA}}$, and $\text{Exp}_{\text{KEM}}^{\text{ROB-CCA}}$.

Theorem 16 (PR-CCA Security of OAEP[†]). Suppose that $\mathcal{F}$ is an η -almost injective function (η -ATIF) family and has rigidity, $\text{SOTP} = (\text{Encode}, \text{Decode}, \mathcal{M}, \mathcal{U}, \mathcal{X}_1, \psi_1)$ is a semi-generalized one-time pad, and ψ_2 is a distribution with collision entropy κ . Then scheme KEM from OAEP[†] in Fig. 5 has PR-CCA security. More precisely, for any PR-CCA adversary $\mathcal{A}$ issuing q_G and q_H (quantum) random oracle queries to G and H , and q_d decryption queries, if $q = q_G + q_H + q_d + 2$, then we have

$$\text{Adv}_{\text{KEM}}^{\text{PR-CCA}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{F}, \psi}^{\text{PR}}(\mathcal{B}) + 16q^2 \sqrt{\text{Adv}_{\mathcal{F}, \psi}^{\text{PD-OW}}(\mathcal{B})} + 4q \sqrt{\frac{300q^3}{2^\lambda} + \frac{16\sqrt{2}q^2}{2^{\lambda/2}} + \frac{6\sqrt{q}}{2^{\kappa/4}}} + \eta.$$

Theorem 17 (ROB-CCA Security of OAEP[†]). If the ATIF family $\mathcal{F}$ and the hash function H are both collision resistant, then scheme KEM from OAEP[†] in Fig. 5 has ROB-CCA security. More precisely, for any ROB-CCA adversary $\mathcal{A}$ issuing q_H (quantum) random oracle queries to H , we have $\text{Adv}_{\text{KEM}}^{\text{ROB-CCA}}(\mathcal{A}) \leq \frac{648(q_H+1)^3}{2^\lambda} + \text{negl}(\lambda)$.

The proofs of the security notions above are provided in Appendix A.

4.4 Expected Security

In this subsection, we discuss the expected bit security of OAEP-NTRU. The security proof in Subsect. 4.1 and 4.3 reduces the security of the scheme to the $\text{NTRU}_{\mathcal{R}, \psi}$ assumption and the computational and decisional $\text{RLWE}_{\mathcal{R}, \psi}$ assumptions, where $\mathcal{R} = \mathbb{Z}_q[X]/(X^n - X^{n/2} + 1)$ and $\psi = (\text{CBD}_1^n, \text{CBD}_1^n)$. We estimate the concrete security of both the NTRU instance

$$h = g(3f' + 1)^{-1}, \quad f', g \leftarrow \text{CBD}_1^n,$$

and the Ring-LWE instance

$$(a, b = as + e), \quad a \leftarrow U(\mathcal{R}), \quad s, e \leftarrow \text{CBD}_1^n,$$

for each parameter set. The estimates are obtained with the Lattice Estimator [5], using the reported $\log_2(\text{rop})$ value as the bit-security estimate. The Lattice Estimator evaluates the best known attacks on LWE/RLWE, including primal, dual, combinatorial, and algebraic attacks, and also provides dedicated estimators for NTRU public-key recovery attacks.

- **Ring-LWE attacks.** For the Ring-LWE assumption we expand the ring instance into the corresponding LWE instance of dimension n with $m = n$ samples and secret/error distribution CBD_1 . We consider the attack families implemented by the Lattice Estimator. The primal attacks include the uSVP embedding attack based on BKZ reduction [15, 6, 4, 16, 32], the BDD attack based on BKZ preprocessing followed by an SVP/CVP step [30], and the primal hybrid attack that guesses part of the secret before solving the remaining BDD instance [9, 27]. We also consider dual attacks, including the plain dual attack and dual-hybrid variants [3, 22].
- **NTRU attacks.** For the $\text{NTRU}_{\mathcal{R},\psi}$ assumption we evaluate attacks on the NTRU lattice associated with the public key h . The relevant attacks are primal attacks: the uSVP attack on the NTRU lattice, the BDD attack [30], and the hybrid Howgrave-Graham style attack that combines lattice reduction with guessing of a subset of secret coordinates [27]. We also consider the meet-in-the-middle hybrid variant implemented by the estimator and the dense-sublattice attack for overstretched NTRU parameters [29, 18]. The latter does not improve the attack cost for our moduli.

The concrete security of our scheme is shown in Table 2. The concrete security of a parameter set is the minimum of the NTRU and Ring-LWE estimates, together with the independent requirements from the collision entropy of CBD_1^n , the injectivity error of $\mathcal{F}_{\text{ntru}}$, and the output length of the hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$.

Table 2: Bit security of the underlying NTRU and Ring-LWE assumptions.

Scheme	Type of Security	Bit Security of NTRU	Bit Security of Ring-LWE
OAEP-NTRU-648 ($n = 648, q = 7129$)	classical security	144	141
	quantum security	138	135
OAEP-NTRU-1296 ($n = 1296, q = 17497$)	classical security	273	262
	quantum security	254	246
OAEP-NTRU-2592 ($n = 2592, q = 28513$)	classical security	512	513
	quantum security	468	469

5 Performance Analysis

5.1 Efficiency Analysis of OAEP-NTRU

The main time consumption in OAEP-NTRU lies in the computation of hash functions and some polynomial arithmetic. The latter mainly includes performing NTT, InvNTT, and the polynomial operations of multiplication and inverse. Some simple operations, such as addition, subtract and multiply by constant numbers, contribute much smaller to the time consumption, so we omit it in this analysis. We always use the polynomial’s image under NTT as its representation when computing multiplication and inverse of polynomials.

Table 3 shows the (expected) number of these operations performed in each algorithms. The “MUL” column shows the number of polynomial multiplication and the “INV” column shows the number of polynomial inversion.

Table 3: Number of operations in OAEP-NTRU.

Algorithm	Hash	NTT	InvNTT	MUL	INV
KeyGen	1	$T_f + T_g \approx 2$	0	1	1
Encap	3	2	0	1	0
Decap	2	1	2	2	0

Here T_f and T_g are the expected number of iterations of generating $\mathbf{f}$ and $\mathbf{g}$ in KeyGen, respectively. Our experimental results show that $T_f \approx T_g \approx 1$, with probability $1 - 2^{-18}$.

5.2 Performance of OAEP-NTRU Implementation

We provide a reference implementation and an optimized implementation with the AVX2 instruction set in our submission package. Table 4 shows the performance evaluation of the two implementations of OAEP-NTRU. $|pk|$, $|ct|$, $|sk|$, $|k|$ represent the byte size of public key, ciphertext, secret key and the encapsulated key, respectively. $T(\text{Alg})$ represents the (average) kilo-cycles of the corresponding algorithm of KEM schemes. The column of “Ref” evaluates the running time of the reference implementation in the submission package, and the column of “AVX2” evaluates that of the optimized implementation. Both implementations directly use the provided libraries for DRNG and hash functions, and share the same known answer tests (KAT) outputs.

The performance evaluations are obtained by experiments on a computer with Intel(R) Core(TM) i7-11700F CPU. The machine has 16GB memory and is running Debian GNU/Linux with Linux kernel version 6.12.90. We compile our implementations with gcc 16.1.1 compiler. The compiler flags are provided in the Makefiles included in the submission package.

For reproducibility of the known answer tests (KAT), both our reference and optimized implementations in the submission package are sticking to the

Table 4: Performance evaluation results (size in bytes and time in kilo-cycles).

Algorithm Name	$ pk $	$ sk $	$ ct $	$ k $	T(KeyGen)		T(Encap)		T(Decap)	
					Ref	AVX2	Ref	AVX2	Ref	AVX2
OAEP-NTRU-648	1053	2138	1085	32	104	40	66	48	53	30
OAEP-NTRU-1296	2430	4924	2494	32	272	111	215	167	146	95
OAEP-NTRU-2592	4860	9848	4988	64	711	256	634	488	500	350

provided library for hash functions and DRNG. Therefore, the optimized implementation only use AVX2 instruction sets on polynomial operations (including, NTT, InvNTT, the polynomial arithmetics and the polynomial encoding/decoding) to improve the efficiency. The implementation of hash functions stays untouched, thus the time consumed by the hash function evaluations is similar to that in the reference implementation. In real-world deployment, the hash function can still be optimized using AVX2 or other SIMD instruction sets, further improving the performance of our algorithm.

5.3 Performance Comparison with Other KEMs

We ran experiments to compare the performance of the reference implementation between Kyber [8], NTRU+ [28] and our OAEP-NTRU. We perform the experiment on the same platform as specified in Subsect. 5.2. Table 5 compares the performance of OAEP-NTRU with Kyber and NTRU+. T(Alg) represents the (average) kilo-cycles of the corresponding algorithm of KEM schemes.

Table 5: Performance comparison with Kyber and NTRU+ (size in bytes and time in kilo-cycles).

Algorithm Name	Target Classical Security	Security Level (bits)		$ pk $	$ sk $	$ ct $	$ k $	T (KeyGen)	T (Encap)	T (Decap)	Correct. Error δ
		Classical	Quantum								
Kyber512	128	118	107	800	1632	768	32	61	71	94	2^{-139}
NTRU+768		156	139	1152	2336	1152	32	83	35	39	2^{-379}
OAEP-NTRU-648		141	135	1053	2138	1085	32	92	59	53	2^{-159}
Kyber1024	256	256	232	1568	3168	1568	32	149	152	188	2^{-174}
NTRU+1152		248	222	1728	3488	1728	32	158	60	69	2^{-260}
OAEP-NTRU-1296		262	246	2430	4924	2494	32	241	200	142	2^{-978}
OAEP-NTRU-2592	512	512	468	4860	9848	4988	64	678	618	500	2^{-652}

For our OAEP-NTRU, we note that the implementation for the known answer tests (KAT) generation requires using the provided DRNG for all random numbers. However, the provided DRNG are implemented using hash chain. This causes a dramatic drawback in its efficiency compared to using the random function provided by the operating system directly, as Kyber and NTRU+

did in their implementation. So for fairness, during this experiment for comparison, we replace the provided pseudorandom generator with `/dev/urandom` on Linux when running performance evaluation for comparison. This approach aligns with the real-world case, where the user decides which random number source to use, and a common decision is to use the system-provided RNG. This version of our implementation is provided as additional implementations in the submission package.

Regarding the correctness error, we note that our notion is stronger than those used for NTRU+ and Kyber. In our case, the correctness error implies that, except for a δ fraction of key pairs, every generated key pair achieves perfect correctness. In contrast, the correctness errors of NTRU+ and Kyber are average-case measures: their decapsulation failure probability is δ when averaged over the randomness of both KeyGen and Encap.

6 Advantages and Limitations

6.1 Advantages

In addition to the efficiency and the security properties, our OAEP-NTRU enjoys unique advantages as follows.

(1) New OAEP[†] Transform. OAEP-NTRU is built from the novel OAEP[†] transform. This is in contrast to many other existing KEM schemes like ML-KEM/Kyber, HQC, NTRU+, which rely on the FO transform. The new OAEP[†] enriches the design paradigm for KEM, and reduces the security risk associated with relying on a single paradigm to construct post-quantum KEMs.

Moreover, OAEP[†] is a generic and flexible transform, admitting many instantiations from various assumptions. In this document, we instantiate OAEP[†] from NTRU lattice problems for better efficiency, yielding the strongly secure and highly efficient OAEP-NTRU.

(2) No Re-Encryption in Decapsulation. OAEP-NTRU does not require a costly and side-channel-vulnerable re-encryption check during decapsulation, which improves both performance and security against side-channel attacks. In contrast, FO-based schemes require a re-encryption step involving multiple complex operations, which makes them more expensive to use for protection against side-channel attacks (cf. Subsect. 6.3 for more explanations).

(3) Security beyond IND-CCA. OAEP-NTRU achieves strong security including ANO-CCA (anonymity), PR-CCA (ciphertext pseudo-randomness) and ROB-CCA (robustness) simultaneously. In comparison, FO-based schemes can hardly achieve robustness as noted in [40] (cf. Table 7 in Subsect. 6.3), and the original OAEP and its variants like OAEP+, Q-OAEP are not known to achieve anonymity and robustness in the QROM (cf. Table 6 in Subsect. 6.2).

(4) Low Decapsulation Failure Probability. OAEP-NTRU provides strong correctness guarantee. For all but a 2^{-159} fraction of key pairs, the decapsu-

lation is perfectly correct: every honestly generated ciphertext can be decapsulated successfully. This is in contrast to many FO-based KEMs like Kyber and NTRU+, where the decryption-failure probability is averaged over both the randomness of key generation and the randomness used in encapsulation/encryption.

(5) New Mode of Partial Message Recovery. Our OAEP-NTRU supports a mode with partial message recovery. In this mode, `Encap` not only encapsulate the key k , but also takes an extra message m as input and encrypt it in the encapsulation ct . With this mode, we can obtain a PKE scheme with a more compact ciphertext using a modified KEM-DEM paradigm.

Specifically, when we are encrypting a long message, say $m = m_1 || m_2 \in \{0, 1\}^{l_m}$ with γ -bit m_1 and $(l_m - \gamma)$ -bit m_2 . We can invoke the `Encap` of the KEM with partial message recovery with m_1 as the extra message, and use the encapsulated key k to encrypt m_2 with the DEM. Compared to using the ordinary KEM-DEM paradigm, this approach saves up to n bits of space, as we can encrypt at most n bits of extra message into the ciphertext.

6.2 Comparison to OAEP, OAEP+ and Q-OAEP

Table 6: Comparison to OAEP and its variants. “type of F, F’s one-wayness, F’s pre-image distribution” denote the requirements for the function F underlying the OAEP variant. “TDP” means trapdoor permutation, “TIF” means trapdoor injective function, and “ATIF” means almost trapdoor injective function. “OW” denotes one-wayness and “PDOW” denotes partial domain one-wayness. “ciphertext overhead” represents the increased ciphertext size compared to the output size of the underlying function F. “CCA”, “ANO”, and “ROB” represent the indistinguishability, anonymity, and robustness under CCA attacks of the PKE schemes from OAEP (and its variants). “inst” denotes whether there exists any instantiation from post-quantum assumptions. “?” denotes unknown.

Transform	type of F	F’s one-wayness	F’s pre-image distribution	ciphertext overhead	CCA	ANO	ROB	inst
OAEP[20]	TDP	PDOW	uniform	0	✓	?	?	?
OAEP+[21]	TDP	OW	uniform	0	✓	?	?	?
Q-OAEP[36]	TIF	PDOW	uniform	≥ 126 bytes	✓	?	?	✓
Our OAEP [†]	ATIF	PDOW	flexible	32 bytes	✓	✓	✓	✓

Compared to OAEP variants, including OAEP, OAEP+ and Q-OAEP, our new OAEP[†] transform enjoys the following advantages:

- (1) OAEP[†] *admits efficient post-quantum instantiations from NTRU*. Compared to the OAEP and OAEP+ transforms, whose building block is a trapdoor permutation (TDP) that has hardly any instantiation from isogenies and lattices, our OAEP[†] admits ATIF which has instantiation from NTRU, allowing post-quantum security of the obtained scheme.

Compared to Q-OAEP, which uses trapdoor injective function (TIF) as building blocks and requires perfect injectivity of TIF, our OAEP[†] requires only overwhelming injectivity for ATIF and support more flexible pre-image distributions for ATIF.

In particular, this allows our OAEP[†] transform to accommodate the trapdoor injective function F_{NTRU} contained in NTRU-HPS-2048677 (a third-round KEM candidate submitted to NIST) [14] and keeps its optimized advantage while helping it to achieve CCA security, pseudo-randomness, anonymity and robustness.

- (2) OAEP[†] achieves CCA security with security decay only $\epsilon^{1/2}$ from ϵ -PDOW of ATIF in the QRO model. The security decay $\epsilon^{1/2}$ is much better than $\epsilon^{1/8}$ in Q-OAEP.
- (3) OAEP[†] avoids an additional hash function and supports flexible pre-image distribution of ATIF. Our OAEP[†] makes use of only two hash functions for leveraging CCA security (no additional hash function involved). Notably, our OAEP[†] is structured with two hash functions in a way differently from OAEP and admits more flexible pre-image distributions for ATIF.

6.3 Comparison to FO-based schemes

Table 7: Comparison to schemes from FO transform

Scheme	IND-CCA	ANO-CCA	PR-CCA	ROB-CCA
Kyber [8]	✓	✓	✓	×
NTRU+ [28]	✓	✓	✓	?
NTRU-HPS [14]	✓	✓	✓	×
Our OAEP-NTRU	✓	✓	✓	✓

Our KEM is based on the OAEP[†] transform, which differs from the commonly used FO transform in the construction of CCA-secure KEMs. This choice gives us advantages as follows:

- (1) OAEP[†] is more side-channel secure. Unlike FO transform, OAEP[†] does not need re-encryption check in the decapsulation algorithm, which improves the decapsulation efficiency and avoids potential side-channel attacks due to re-encryption [38].

More precisely, recall than in the FO transform, its re-encryption step involves multiple complex operations (re-generating error vectors, polynomial arithmetic, and NTT transformations), making it more vulnerable to side-channel attacks (like [25, 33]). Accordingly, it is expensive to protect the re-encryption step in FO against side-channel attacks.

In contrast, in OAEP[†], the key confirmation step only performs a hash comparison (which checks whether the hash value of H begins with

σ'). Therefore, it is easier to resist side-channel attacks in this step, e.g., by using a side-channel secure hash function [34, 37] and a constant-time string comparison.

- (2) Compared to the NIST standardized KEM scheme Kyber512 [8], our $\text{KEM}_{\text{NTRU}}^1$ is faster. $\text{KEM}_{\text{NTRU}}^2$ shares a same basic underlying NTRU as NTRU-hps2048677, keeps optimized parameters of the basic NTRU and is comparable to the FO-variant NTRU-hps2048677.
- (3) Compared to NTRU+KEM576 and NTRU+PKE576, the final scheme of Korean PQC competition [2], our $\text{KEM}_{\text{NTRU}}^1$ and $\text{KEM}_{\text{NTRU}}^2$ are a little bit slower. However, the security proofs of NTRU+KEM576 and NTRU+PKE576 rely on computational injectivity of $\mathbb{F}$, and has a security gap as pointed out in [11] (See Sect. 3.4.2 in [11]). This gap was admitted by its submitter [1].

6.4 Limitations

A limitation of our OAEP-NTRU is that its strong correctness guarantee pays a price of slightly fast growing parameters. Our design needs every honestly generated ciphertext decrypts correctly except for a negligible fraction of key pairs. This results in strong correctness, which is better than the usual average-case correctness, but makes parameter selection more conservative. In particular, as the target security level increases, the lattice dimension and modulus need to grow faster in order to simultaneously guarantee hardness and strong correctness. This leads to larger public keys and ciphertexts than those schemes with average-case correctness. One possible way to relax this limitation is to adopt the average-case correctness notion. In this way, one can gain better efficiency for OAEP-NTRU with smaller parameters while preserving the same security levels.

References

- [1] Kpqc-bulletin (2024), <https://groups.google.com/g/kpqc-bulletin/c/LC9IpjasXac>
- [2] Kpqc competition (2025), https://www.kpqc.or.kr/competition_02.html
- [3] Albrecht, M.R.: On dual lattice attacks against small-secret LWE and parameter choices in HELib and SEAL. In: Coron, J.S., Nielsen, J.B. (eds.) EUROCRYPT 2017, Part II. LNCS, vol. 10211, pp. 103–129. Springer, Cham (Apr / May 2017). https://doi.org/10.1007/978-3-319-56614-6_4
- [4] Albrecht, M.R., Göpfert, F., Virdia, F., Wunderer, T.: Revisiting the expected cost of solving uSVP and applications to LWE. In: Takagi,

- T., Peyrin, T. (eds.) ASIACRYPT 2017, Part I. LNCS, vol. 10624, pp. 297–322. Springer, Cham (Dec 2017). https://doi.org/10.1007/978-3-319-70694-8_11
- [5] Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. *J. Math. Cryptol.* **9**(3), 169–203 (2015), <http://www.degruyter.com/view/j/jmc.2015.9.issue-3/jmc-2015-0016/jmc-2015-0016.xml>
 - [6] Alkim, E., Ducas, L., Pöppelmann, T., Schwabe, P.: Post-quantum key exchange - A new hope. In: Holz, T., Savage, S. (eds.) *USENIX Security 2016*. pp. 327–343. USENIX Association (Aug 2016)
 - [7] Ambainis, A., Hamburg, M., Unruh, D.: Quantum security proofs using semi-classical oracles. In: Boldyreva, A., Micciancio, D. (eds.) *CRYPTO 2019, Part II*. LNCS, vol. 11693, pp. 269–295. Springer, Cham (Aug 2019). https://doi.org/10.1007/978-3-030-26951-7_10
 - [8] Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Kyber algorithm specifications and supporting documentation <https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf>
 - [9] Bai, S., Galbraith, S.D.: Lattice decoding attacks on binary LWE. In: Susilo, W., Mu, Y. (eds.) *ACISP 14*. LNCS, vol. 8544, pp. 322–337. Springer, Cham (Jul 2014). https://doi.org/10.1007/978-3-319-08344-5_21
 - [10] Bellare, M., Rogaway, P.: Optimal asymmetric encryption. In: Santis, A.D. (ed.) *EUROCRYPT’94*. LNCS, vol. 950, pp. 92–111. Springer, Berlin, Heidelberg (May 1995). <https://doi.org/10.1007/BFb0053428>
 - [11] Bernstein, D.J., Cottaar, J., Giandomenico, E.D., Hövelmanns, K., Hülsing, A., Kudinov, M., Lange, T., Mahzoun, M., Meijers, M., Pellegrini, A., Ravagnani, A., Ritsch, S., Schäge, S., Tang, T., Trimoska, M., Vorstermans, M., Weber, F.J.: Report on evaluation of KpqC round-2 candidates. *Cryptology ePrint Archive*, Paper 2024/2077 (2024), <https://eprint.iacr.org/2024/2077>
 - [12] Bernstein, D.J., Persichetti, E.: Towards KEM unification. *Cryptology ePrint Archive*, Report 2018/526 (2018), <https://eprint.iacr.org/2018/526>
 - [13] Bjørstad, T.E., Dent, A.W., Smart, N.P.: Efficient kems with partial message recovery. In: Galbraith, S.D. (ed.) *Cryptography and Coding*, 11th IMA International Conference, Cirencester, UK, December 18–20, 2007, Proceedings. *Lecture Notes in Computer Science*, vol. 4887, pp. 233–256. Springer (2007). https://doi.org/10.1007/978-3-540-77272-9_15, https://doi.org/10.1007/978-3-540-77272-9_15

- [14] Chen, C., Danba, O., Hoffstein, J., Hülsing, A., Rijneveld, J., Schanck, J.M., Schwabe, P., Whyte, W., Zhang, Z.: Algorithm specifications and supporting documentation. Brown University and Onboard security company, Wilmington USA (2019), <https://ntru.org>.
- [15] Chen, Y., Nguyen, P.Q.: BKZ 2.0: Better lattice security estimates. In: Lee, D.H., Wang, X. (eds.) ASIACRYPT 2011. LNCS, vol. 7073, pp. 1–20. Springer, Berlin, Heidelberg (Dec 2011). https://doi.org/10.1007/978-3-642-25385-0_1
- [16] Dachman-Soled, D., Ducas, L., Gong, H., Rossi, M.: LWE with side information: Attacks and concrete security estimation. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020, Part II. LNCS, vol. 12171, pp. 329–358. Springer, Cham (Aug 2020). https://doi.org/10.1007/978-3-030-56880-1_12
- [17] Don, J., Fehr, S., Majenz, C., Schaffner, C.: Online-extractability in the quantum random-oracle model. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part III. LNCS, vol. 13277, pp. 677–706. Springer, Cham (May / Jun 2022). https://doi.org/10.1007/978-3-031-07082-2_24
- [18] Ducas, L., van Woerden, W.P.J.: NTRU fatigue: How stretched is over-stretched? In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021, Part IV. LNCS, vol. 13093, pp. 3–32. Springer (2021). https://doi.org/10.1007/978-3-030-92068-5_1
- [19] Duman, J., Hövelmanns, K., Kiltz, E., Lyubashevsky, V., Seiler, G., Unruh, D.: A thorough treatment of highly-efficient NTRU instantiations. In: Boldyreva, A., Kolesnikov, V. (eds.) PKC 2023, Part I. LNCS, vol. 13940, pp. 65–94. Springer, Cham (May 2023). https://doi.org/10.1007/978-3-031-31368-4_3
- [20] Ebrahimi, E.: Post-quantum security of plain OAEP transform. In: Hanaoka, G., Shikata, J., Watanabe, Y. (eds.) PKC 2022, Part I. LNCS, vol. 13177, pp. 34–51. Springer, Cham (Mar 2022). https://doi.org/10.1007/978-3-030-97121-2_2
- [21] Ebrahimi, E.: From plaintext-extractability to IND-CCA security. In: Smith, B., Wu, H. (eds.) SAC 2022. LNCS, vol. 13742, pp. 419–436. Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-58411-4_19
- [22] Espitau, T., Joux, A., Kharchenko, N.: On a dual/hybrid approach to small secret LWE - A dual/enumeration technique for learning with errors and application to security estimates of FHE schemes. In: Bhargavan, K., Oswald, E., Prabhakaran, M. (eds.) INDOCRYPT 2020. LNCS, vol. 12578, pp. 440–462. Springer, Cham (Dec 2020). https://doi.org/10.1007/978-3-030-65277-7_20

- [23] Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. In: Wiener, M.J. (ed.) CRYPTO'99. LNCS, vol. 1666, pp. 537–554. Springer, Berlin, Heidelberg (Aug 1999). https://doi.org/10.1007/3-540-48405-1_34
- [24] Gaborit, P., Aguilar-Melchor, C., Aragon, N., Bettaieb, S., Bidoux, L., Blazy, O., Deneuville, J.C., Persichetti, E., Zémor, G., Bos, J., et al.: Hamming quasi-cyclic (HQC) (2025)
- [25] Guo, Q., Johansson, T., Nilsson, A.: A key-recovery timing attack on post-quantum primitives using the Fujisaki-Okamoto transformation and its application on FrodoKEM. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020, Part II. LNCS, vol. 12171, pp. 359–386. Springer, Cham (Aug 2020). https://doi.org/10.1007/978-3-030-56880-1_13
- [26] Hoffstein, J., Pipher, J., Silverman, J.H.: NTRU: A ring-based public key cryptosystem. In: Buhler, J. (ed.) Algorithmic Number Theory, Third International Symposium, ANTS-III, Portland, Oregon, USA, June 21–25, 1998, Proceedings. Lecture Notes in Computer Science, vol. 1423, pp. 267–288. Springer (1998). <https://doi.org/10.1007/BFb0054868>, <https://doi.org/10.1007/BFb0054868>
- [27] Howgrave-Graham, N.: A hybrid lattice-reduction and meet-in-the-middle attack against NTRU. In: Menezes, A. (ed.) CRYPTO 2007. LNCS, vol. 4622, pp. 150–169. Springer, Berlin, Heidelberg (Aug 2007). https://doi.org/10.1007/978-3-540-74143-5_9
- [28] Kim, J., Park, J.H.: NTRU+: Compact construction of ntru using simple encoding method (2022), <https://sites.google.com/view/ntruplus>
- [29] Kirchner, P., Fouque, P.A.: Revisiting lattice attacks on overstretched NTRU parameters. In: Coron, J.S., Nielsen, J.B. (eds.) EURO-CRYPT 2017, Part I. LNCS, vol. 10210, pp. 3–26. Springer, Cham (Apr / May 2017). https://doi.org/10.1007/978-3-319-56620-7_1
- [30] Liu, M., Nguyen, P.Q.: Solving BDD by enumeration: An update. In: Dawson, E. (ed.) CT-RSA 2013. LNCS, vol. 7779, pp. 293–309. Springer, Berlin, Heidelberg (Feb / Mar 2013). https://doi.org/10.1007/978-3-642-36095-4_19
- [31] Lyubashevsky, V., Seiler, G.: NTTRU: Truly fast NTRU using NTT. IACR TCHES **2019**(3), 180–201 (2019). <https://doi.org/10.13154/tches.v2019.i3.180-201>, <https://tches.iacr.org/index.php/TCHES/article/view/8293>
- [32] Postlethwaite, E.W., Virdia, F.: On the success probability of solving unique SVP via BKZ. In: Garay, J. (ed.) PKC 2021, Part I. LNCS, vol. 12710, pp. 68–98. Springer, Cham (May 2021). https://doi.org/10.1007/978-3-030-75245-3_4

- [33] Ravi, P., Roy, S.S., Chattopadhyay, A., Bhasin, S.: Generic side-channel attacks on CCA-secure lattice-based PKE and KEMs. *IACR TCHES* **2020**(3), 307–335 (2020). <https://doi.org/10.13154/tches.v2020.i3.307-335>, <https://tches.iacr.org/index.php/TCHES/article/view/8592>
- [34] Schwabe, P., Stoffelen, K.: All the AES you need on Cortex-M3 and M4. In: Avanzi, R., Heys, H.M. (eds.) *SAC 2016*. LNCS, vol. 10532, pp. 180–194. Springer, Cham (Aug 2016). https://doi.org/10.1007/978-3-319-69453-5_10
- [35] Shoup, V.: OAEP reconsidered. In: Kilian, J. (ed.) *CRYPTO 2001*. LNCS, vol. 2139, pp. 239–259. Springer, Berlin, Heidelberg (Aug 2001). https://doi.org/10.1007/3-540-44647-8_15
- [36] Targhi, E.E., Unruh, D.: Post-quantum security of the Fujisaki-Okamoto and OAEP transforms. In: Hirt, M., Smith, A.D. (eds.) *TCC 2016-B, Part II*. LNCS, vol. 9986, pp. 192–216. Springer, Berlin, Heidelberg (Oct / Nov 2016). https://doi.org/10.1007/978-3-662-53644-5_8
- [37] Ueno, R., Homma, N., Aoki, T.: Toward more efficient DPA-resistant AES hardware architecture based on threshold implementation. In: Guilley, S. (ed.) *COSADE 2017*. LNCS, vol. 10348, pp. 50–64. Springer, Cham (Apr 2017). https://doi.org/10.1007/978-3-319-64647-3_4
- [38] Ueno, R., Xagawa, K., Tanaka, Y., Ito, A., Takahashi, J., Homma, N.: Curse of re-encryption: A generic power/EM analysis on post-quantum KEMs. *IACR TCHES* **2022**(1), 296–322 (2022). <https://doi.org/10.46586/tches.v2022.i1.296-322>
- [39] Unruh, D.: Non-interactive zero-knowledge proofs in the quantum random oracle model. In: Oswald, E., Fischlin, M. (eds.) *EUROCRYPT 2015, Part II*. LNCS, vol. 9057, pp. 755–784. Springer, Berlin, Heidelberg (Apr 2015). https://doi.org/10.1007/978-3-662-46803-6_25
- [40] Xagawa, K.: Anonymity of NIST PQC round 3 KEMs. In: Dunkelman, O., Dziembowski, S. (eds.) *EUROCRYPT 2022, Part III*. LNCS, vol. 13277, pp. 551–581. Springer, Cham (May / Jun 2022). https://doi.org/10.1007/978-3-031-07082-2_20
- [41] Zhandry, M.: A note on the quantum collision and set equality problems. *Quantum Inf. Comput.* **15**(7&8), 557–567 (2015). <https://doi.org/10.26421/QIC15.7-8-2>, <https://doi.org/10.26421/QIC15.7-8-2>

A Security Proofs for OAEP[†]

In this appendix, we provide security proofs of the security properties of the OAEP[†] transform in QROM.

A.1 Lemmas for Quantum Random Oracles

In the quantum world, a quantum algorithm $\mathcal{A}$ can perform superposition queries to the random oracle H , and then oracle H behaves as a unitary operation $|x\rangle|y\rangle \mapsto |x\rangle|y \oplus H(x)\rangle$. In this case, H becomes a quantum random oracle.

We recall the online-extractable technique [17]. First, we define $\Gamma(f)$ and $\Gamma'(f)$ which are associated with a function f .

Definition 18 ([17]). Let $f : \mathcal{X} \times \{0, 1\}^n \rightarrow \mathcal{C}$ be an arbitrary fixed function. Define $\Gamma(f) := \max_{x, c} |\{y \mid f(x, y) = c\}|$, $\Gamma'(f) := \max_{x \neq x', y'} |\{y \mid f(x, y) = f(x', y')\}|$.

Roughly speaking, the online-extractable technique in [17] suggests that a random oracle $H(\cdot)$ can be successfully simulated and the pre-image x can be successfully extracted from the image $c = f(x, H(x))$ by an efficient extractable random oracle simulator $\text{eRO} = (\text{eRO.RO}, \text{eRO.E})$. The following theorem makes the statement formal.

Theorem 19 (Theorem 4.3 in [17]). *Let H be a random oracle, and $\Gamma(\cdot)$ and $\Gamma'(\cdot)$ be defined as per Def. 18. There exists an efficient extractable random oracle simulator $\text{eRO} = (\text{eRO.RO}, \text{eRO.E})$ satisfying the following properties.*

- (1) *If eRO.E is unused, eRO.RO is perfectly indistinguishable from the random oracle $H : \mathcal{X} \rightarrow \{0, 1\}^n$.*
- (2.a) *Any two subsequent independent queries to eRO.RO commute. We refer two subsequent queries as being independent if the input to one query does not depend on the output of the other.*
- (2.b) *Any two subsequent independent queries to eRO.E commute.*
- (2.c) *Any two subsequent independent queries to eRO.E and eRO.RO are $8\sqrt{2\Gamma(f)/2^n}$ -almost-commute.*
- (3) *(Proposition 4.5 in [17].) Consider a query algorithm $\mathcal{A}$ that makes q queries to eRO.RO but no query to eRO.E , outputting some $(c_1, \dots, c_\ell) \in \mathcal{C}^\ell$ and $(x_1, \dots, x_\ell) \in \mathcal{X}^\ell$. Suppose that $(y_1, \dots, y_\ell)$ is obtained by making ℓ additional queries to eRO.RO on input $(x_1, \dots, x_\ell)$, and $(\hat{x}_1, \dots, \hat{x}_\ell)$ obtained by making ℓ additional queries to $\text{eRO.E}(f, c_i)$ for each $i \in [\ell]$. Then*

$$\Pr_{\substack{(c_1, x_1, \dots, c_\ell, x_\ell) \leftarrow \mathcal{A}^{\text{eRO.RO}}, \\ y_i \leftarrow \text{eRO.RO}(x_i), \\ \hat{x}_i \leftarrow \text{eRO.E}(f, c_i)}} [\exists i : \hat{x}_i \neq x_i \wedge f(x_i, y_i) = c_i] \leq \frac{300(q + \ell + 1)^3 \Gamma'(f) + 2}{2^n}. \quad (1)$$

Furthermore, according to (2.c), the above ℓ queries to **eRO.RO** and the ℓ independent queries to **eRO.E** can be arbitrarily interleaved, resulting in $8\ell^2 \sqrt{\frac{2\Gamma(f)}{2^n}}$ argument in the right-side of (1).

Next, let us recall the semi-classical oracle and the punctured oracle, and then recall the semi-classical O2H lemma proposed in [7].

Definition 20 (Semi-Classical Oracle and Punctured Oracle). Let $f_S(x)$ be an indicator function for a set S , which outputs 1 if $x \in S$ and 0 otherwise. A semi-classical oracle $\mathcal{O}_S^{SC}$ for a set S is defined as follows: on input $|x\rangle$, it computes and measures $f_S(x)$ in an auxiliary qubit $|f_S(x)\rangle$ indicating whether $x \in S$ or not. For an oracle H , the punctured oracle $H \setminus S$ is oracle H punctured on S , which upon an input x first queries $\mathcal{O}_S^{SC}(x)$ and then queries $H(x)$.

Lemma 21 (Semi-Classical O2H Lemma, Part of Theorem 1 in [7]). *Let S be a random subset of $\mathcal{X}$. Let $G, H : \mathcal{X} \rightarrow \mathcal{Y}$ be random oracles satisfying $G(x) = H(x)$ for all $x \notin S$. Let z be a random bit-string such that S, G, H, z may have arbitrary joint distribution. Let $\mathcal{A}^{\mathcal{O}}$ be a quantum oracle algorithm which makes at most q queries to oracle $\mathcal{O}$. In the execution of $\mathcal{A}^{G \setminus S}$, let **Find** be the event that any of measurements of $|f_S(x)\rangle$ returns 1. Define*

$$\begin{aligned} P_{\text{left}} &:= \Pr[b = 1 : b \leftarrow \mathcal{A}^H(z)], \\ P_{\text{right}} &:= \Pr[b = 1 \wedge \neg \text{Find} : b \leftarrow \mathcal{A}^{G \setminus S}(z)], \quad P'_{\text{right}} := \Pr[b = 1 : b \leftarrow \mathcal{A}^G(z)], \\ P_{\text{find}} &:= \Pr[\text{Find} : \mathcal{A}^{G \setminus S}(z)]. \end{aligned}$$

$$\begin{aligned} \text{Then } |P_{\text{left}} - P_{\text{right}}| &\leq 2\sqrt{(q+1) \cdot P_{\text{find}}}, \quad |P_{\text{left}} - P'_{\text{right}}| \leq 2\sqrt{(q+1) \cdot P_{\text{find}}} \\ \text{and } |\sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}}| &\leq 2\sqrt{(q+1) \cdot P_{\text{find}}}, \quad |\sqrt{P_{\text{left}}} - \sqrt{P'_{\text{right}}}| \leq 2\sqrt{(q+1) \cdot P_{\text{find}}}. \end{aligned}$$

Lemma 22 (Search in Semi-Classical Oracle, Theorem 2 in [7]). *Let $\mathcal{O}_S^{SC}$ be a semi-classical oracle with domain $\mathcal{X}$. Then $\mathcal{O}_{\emptyset}^{SC}$ is just a quantum oracle. Let $S \subseteq \mathcal{X}$ and $z \in \{0, 1\}^*$ such that S, z may have arbitrary joint distribution. Let $\mathcal{A}^{\mathcal{O}_S^{SC}}(z)$ be any quantum oracle algorithm making at most q queries to $\mathcal{O}_S^{SC}$.*

Let $\mathcal{B}$ be an algorithm that, on input z , chooses $i \leftarrow_{\$} [q]$, runs $\mathcal{A}^{\mathcal{O}_S^{SC}}(z)$ until (just before) the i -th query, and then measures all query input registers in the computational basis and outputs the set T of measurement outcomes. Then

$$\Pr[\text{Find} : \mathcal{A}^{\mathcal{O}_S^{SC}}(z)] \leq 4q \cdot \Pr[S \cap T \neq \emptyset : T \leftarrow \mathcal{B}(z)].$$

Next we recall the adaptive version of O2H lemma. Compared to the semi-classical O2H lemma where the punctured set S is fixed beforehand, the quantum adversary may have interactions with a classical algorithm $\mathcal{C}$, so it can adaptively choose a punctured point $\bar{x}$ based on what it has learned from the interactions. This adaptive O2H lemma was originally proposed for the design of non-interactive zero-knowledge proof and signature schemes in QROM [39].

Lemma 23 (Adaptive O2H Lemma, Lemma 9 in [39]). *Let $H : \{0,1\}^* \rightarrow \{0,1\}^n$ be a random oracle. Consider the following algorithms:*

- *The oracle algorithm $\mathcal{C}_0^H$, that makes at most q_0 (quantum) queries to H .*
- *The classical algorithm $\bar{\mathcal{C}}$, that may access the classical part of the final state of $\mathcal{C}_0^H$, and has output of collision entropy at least k for every initial state.*
- *The oracle algorithm $\mathcal{C}_1^H$, that may access the final states of $\mathcal{C}_0^H$ and $\bar{\mathcal{C}}$ and makes at most q_1 (quantum) queries to H .*
- *Let $\mathcal{C}'^H$ be an oracle algorithm, that on input $(j, B, \bar{x})$ does the following: run $\mathcal{C}_1^H(st, \bar{x}, B)$ until (just before) the j -th query, measure the argument of the query in the computational basis, output the measurement outcome. (When $\mathcal{C}_1^H(st, \bar{x}, B)$ makes less than j queries, $\mathcal{C}'^H$ outputs $\perp$.)*

Let $P_A^1 := \Pr [\beta = 1 : (\bar{m}, st) \leftarrow \mathcal{C}_0^H, \bar{x} \leftarrow \bar{\mathcal{C}}(\bar{m}), \beta \leftarrow \mathcal{C}_1^H(st, \bar{x}, H(\bar{x}))]$,
 $P_A^2 := \Pr [\beta = 1 : (\bar{m}, st) \leftarrow \mathcal{C}_0^H, \bar{x} \leftarrow \bar{\mathcal{C}}(\bar{m}), B \leftarrow_{\$} \{0,1\}^n, \beta \leftarrow \mathcal{C}_1^H(st, \bar{x}, B)]$,
 $P_C := \Pr [\bar{x} = \bar{x}' : (\bar{m}, st) \leftarrow \mathcal{C}_0^H, \bar{x} \leftarrow_{\$} \bar{\mathcal{C}}(\bar{m}), B \leftarrow_{\$} \{0,1\}^n, j \leftarrow_{\$} [q_1], \bar{x}' \leftarrow \mathcal{C}'^H(j, B, \bar{x})]$.
Then $|P_A^1 - P_A^2| \leq 2q_1\sqrt{P_C} + (4 + \sqrt{2})\sqrt{q_0}2^{-k/4}$.

Lemma 24 (Theorem 3.1 in [41]). *Let $\mathcal{X}$ and $\mathcal{Y}$ be finite sets. Let $H : \mathcal{X} \rightarrow \mathcal{Y}$ be a random oracle. If an unbounded-time quantum adversary $\mathcal{A}$ makes a (quantum) query to H at most q times, then we have*

$$\Pr [H(x_0) = H(x_1) \wedge x_0 \neq x_1 : (x_0, x_1) \leftarrow \mathcal{A}^H] \leq \frac{648(q+1)^3}{|\mathcal{Y}|}.$$

A.2 Proof of IND-CCA Security

Proof of Theorem 15. We now consider a sequence of games for a *fixed* key pair (F, F^{-1}) and analyze $\mathcal{A}$'s advantages in these games. Define $G_i \Rightarrow 1$ as the event $b = b'$ in G_i .

Let $\text{Adv}_{F, F^{-1}}^{\text{CCA}}(\mathcal{A})$ be $\mathcal{A}$'s advantage for the key pair (F, F^{-1}) . We know that $\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}) = \mathbb{E}_{F, F^{-1}} [\text{Adv}_{F, F^{-1}}^{\text{CCA}}(\mathcal{A})]$. Note that $\text{Adv}_{F, F^{-1}}^{\text{CCA}}(\mathcal{A}) \leq 1$ for all $(F, F^{-1}) \leftarrow \text{Gen}$. Due to the η -injective error of $\mathcal{F}$, we have

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq \mathbb{E}_{\text{injective } F, F^{-1}} [\text{Adv}_{F, F^{-1}}^{\text{CCA}}(\mathcal{A})] + \eta.$$

In the following games we focus on $\mathcal{A}$'s advantage on those injective pairs (F, F^{-1}) , which satisfies $F^{-1}(F(x_1, x_2)) = (x_1, x_2)$ for all $(x_1, x_2) \in \mathcal{X}_1 \times \mathcal{X}_2$.

The proof of Theorem 15 proceeds via games G_0 - G_6 and showing $|\Pr[G_i \Rightarrow 1] - \Pr[G_{i-1} \Rightarrow 1]| \leq \text{negl}(\lambda)$ for $i \in [6]$. The pseudo-codes of G_0 - G_6 are shown in Fig. 8. To prove $|\Pr[G_3 \Rightarrow 1] - \Pr[G_2 \Rightarrow 1]| \leq \text{negl}(\lambda)$, we further consider games from $G_3 = G_{3.1}$ to $G_{3.8}$ whose pseudo-codes are shown in Fig. 9.

GAME SETUP G_0 - G_6	$DEC_{ct^*}(ct)$ G_0 - G_6
01: $(pk, sk) := (F, F^{-1})$	17: If $ct = ct^*$: Return $\perp$
02: $b \leftarrow_s \{0, 1\}$	18: Parse $ct^* := (c^*, \sigma^*)$
03: $x_1^* \leftarrow \psi_1$	19: Parse $ct := (c, \sigma)$
04: $x_2^* \leftarrow \psi_2$	20: If $c = c^*$: Return $\perp$
05: $(m_0, m_1, st) \leftarrow \mathcal{A}^{G, H, \text{DECAP}^\perp}(pk)$	21: $(x_1, x_2) := F^{-1}(c)$
06: $(m_0, m_1, st) \leftarrow \mathcal{A}^{G, \text{eRO}, \text{RO}, \text{DECAP}^\perp}(pk)$	22: $(pk', x_1, x_2) := \text{eRO.E}(f, \sigma)$
07: $(m_0, m_1, st) \leftarrow \mathcal{A}^{\bar{G} \setminus \{x_2^*\}, \text{eRO}, \text{RO}, \text{DECAP}^\perp}(pk)$	23: If $pk' \neq pk$: Return $\perp$
08: $x_1^* := \text{Encode}(m_b, G(x_2^*))$	24: If $(x_1, x_2) = \perp$: Return $\perp$
09: $c^* := F(x_1^*, x_2^*)$	25: If $F(x_1, x_2) \neq c$: Return $\perp$
10: $(\sigma^*, k^*) := H(pk, x_1^*, x_2^*)$	26: $(\sigma', k) := H(pk, x_1, x_2)$
11: $\sigma^* \leftarrow_s \{0, 1\}^\lambda, k^* \leftarrow_s \{0, 1\}^\lambda$	27: $(\sigma', k) := \text{eRO.RD}(pk, x_1, x_2)$
12: $ct^* := (c^*, \sigma^*), k_b^* := k^*, k_1^* \leftarrow_s \{0, 1\}^\lambda$	28: If $\sigma' \neq \sigma$: Return $\perp$
13: $b' \leftarrow \mathcal{A}^{G, H, \text{DECAP}_{ct^*}}(st, ct^*, k_b^*)$	29: $r := G(x_2)$
14: $b' \leftarrow \mathcal{A}^{G, \text{eRO}, \text{RO}, \text{DECAP}_{ct^*}}(st, ct^*, k_b^*)$	30: $r := G(x_2)$
15: $b' \leftarrow \mathcal{A}^{\bar{G} \setminus \{x_2^*\}, \text{eRO}, \text{RO}, \text{DECAP}_{ct^*}}(st, ct^*, k_b^*)$	31: If $x_2 = x_2^*$: Find $r := 1$
16: Return $b = b'$	32: $m := \text{Decode}(x_1, r)$
	33: Return (m, k)

Figure 8: G_0 - G_6 for the proof of Theorem 15. If a code line has no “//”, this line occurs in all games G_0 - G_6 .

Game G_0 . This is the original IND-CCA experiment for a fixed (F, F^{-1}) , denoted by $\text{Exp}_{\text{KEM}}^{\text{IND-CCA}}(F, F^{-1})$, except that the challenger $\mathcal{C}$ chooses $x_2^* \leftarrow \psi_2$ at the beginning of the game. This is a conceptual change, so we have

$$\text{Adv}_{F, F^{-1}}^{\text{CCA}}(\mathcal{A}) = \left| \Pr \left[\text{Exp}_{\text{KEM}}^{\text{IND-CCA}}(F, F^{-1}) \Rightarrow 1 \right] - 1/2 \right| = |\Pr[G_0 \Rightarrow 1] - 1/2|.$$

Game G_1 . G_1 is the same as G_0 , except that the challenger $\mathcal{C}$ adds a new rejection rule in DECAP oracle after $\mathcal{A}$ obtains the challenge ciphertext $ct^* = (c^*, \sigma^*)$. In G_1 , the new rejection rule in DECAP is as follows.

- **Rejection rule:** $\text{DECAP}(ct = (c, \sigma))$ will return $\perp$ if $c = c^*$.

G_1 and G_0 differs only when $\mathcal{A}$ makes a decryption query $ct = (c, \sigma)$ such that $c = c^*$ and $\sigma \neq \sigma^*$, but $\text{DECAP}(ct = (c^*, \sigma))$ in G_0 does not return $\perp$. This is impossible due to the injectivity of F , since $(x_1^*, x_2^*) = F^{-1}(c^*) = F^{-1}(c) = (x_1, x_2)$ implies $(\sigma, k) = H(pk, x_1, x_2) = H(pk, x_1^*, x_2^*) = (\sigma^*, k^*)$. So we have

$$\Pr[G_1 \Rightarrow 1] = \Pr[G_0 \Rightarrow 1].$$

Game G_2 . G_2 is the same as G_1 , except that the challenger $\mathcal{C}$ changes the check rule in DECAP oracle. In G_2 , for $\mathcal{A}$'s decryption query $\text{DECAP}(ct = (c, \sigma))$, after $\mathcal{C}$ computes $(x_1, x_2) := F^{-1}(c)$, $\mathcal{C}$ will check whether $c = F(x_1, x_2)$ rather than check whether $(x_1, x_2) = \perp$ as did in G_1 . Note that if $(x_1, x_2) = \perp$, it must hold $c \neq F(x_1, x_2)$. So G_2 and G_1 differ only when $(x_1, x_2) := F^{-1}(c) \neq \perp$ but $F(x_1, x_2) \neq c$. By the rigidity of the underlying F , this event does not happen. So we have $\Pr[G_2 \Rightarrow 1] = \Pr[G_1 \Rightarrow 1]$.

Game G_3 . G_3 is the same as G_2 , except for the generation of (σ^*, k^*) in the challenge phase. In G_3 , after $\mathcal{A}$ submits (m_0, m_1) , $\mathcal{C}$ samples $(\sigma^*, k^*) \leftarrow_s \{0, 1\}^{2\lambda}$, rather than computes $(\sigma^*, k^*) := H(pk, x_1^*, x_2^*)$ as in G_2 .

Now we bound the distinguishing advantage of G_3 and G_2 using the adaptive O2H lemma (cf. Lemma 23).

Let $\mathcal{C}_0^H$ be the challenger algorithm $\mathcal{C}$ in the pre-challenge phase in G_2/G_3 . That is, $\mathcal{C}_0^H$ samples $x_2^* \leftarrow \psi_2$, samples $b \leftarrow \{0, 1\}$, invokes $(m_0, m_1, st) \leftarrow \mathcal{A}(pk = F)$, and returns $\bar{m} := (x_2^*, m_b)$. Let $\bar{\mathcal{C}}$ be a part of the challenger algorithm in the challenge phase. That is, $\bar{\mathcal{C}}(x_2^*, m_b)$ computes $x_1^* = \text{Encode}(m_b, G(x_2^*))$, $c^* = F(x_1^*, x_2^*)$ and outputs $\bar{x} := (pk, x_1^*, x_2^*)$. Define $\mathcal{C}_1^H(st, \bar{x} = (pk, x_1^*, x_2^*), (\sigma^*, k^*) = H(pk, x_1^*, x_2^*)$ or $(\sigma^*, k^*) \leftarrow_{\$} \{0, 1\}^{2\lambda}$) as the rest of $\mathcal{C}$ in G_2/G_3 : $\mathcal{C}_1^H$ takes (σ^*, k^*) as input, obtains $ct^* = (c^*, \sigma^*)$ and $k_0^* := k^*$, and then $\mathcal{C}_1^H$ behaves like the part of $\mathcal{C}$ in the after-challenge phase in G_2 . That is, $\mathcal{C}_1^H$ invokes $b' \leftarrow \mathcal{A}(ct^* = (c^*, \sigma^*), k_b^*)$ and outputs $\beta := 1$ if $b = b'$ and $\beta := 0$ if $b \neq b'$.

It is easy to see that $(\mathcal{C}_0^H, \bar{\mathcal{C}}, \mathcal{C}_1^H)$ is just G_2 if $(\sigma^*, k^*) = H(pk, x_1^*, x_2^*)$ and it is just G_3 if $(\sigma^*, k^*) \leftarrow_{\$} \{0, 1\}^{2\lambda}$. Here $\mathcal{C}_0^H, \mathcal{C}_1^H$ use oracle H to answer all the H queries from $\mathcal{A}$ and the decryption oracle. Suppose $\mathcal{A}$ makes q'_H (so $q'_H \leq q_H$) queries to H after it obtains ct^* . Let $P_A^1 := \Pr[G_2 \Rightarrow 1]$ and $P_A^2 := \Pr[G_3 \Rightarrow 1]$. By Lemma 23, we have $|\Pr[G_3 \Rightarrow 1] - \Pr[G_2 \Rightarrow 1]|$

$$\leq 2q'_H \sqrt{\Pr[(\hat{x}_1, \hat{x}_2) = (x_1^*, x_2^*) : (\hat{x}_1, \hat{x}_2) \leftarrow \mathcal{C}' \text{ in } G_3]} + \frac{6\sqrt{q_H - q'_H}}{2^{\kappa/4}}, \quad (2)$$

where $\mathcal{C}'$ behaves just like $\mathcal{C}$ in G_3 but measures $\mathcal{A}$'s j -th (post-challenge) query to H after $\mathcal{A}$ obtains (ct^*, k^*) and outputs the measurement outcome for $j \leftarrow_{\$} [q'_H]$.

Note that beyond $\mathcal{A}$'s direct hash queries to H , there are also DECAP oracle's queries to $H(pk, x_1, x_2)$ resulting from $\mathcal{A}$'s $\text{DECAP}(c, \sigma)$ queries and the intermediate values $(x_1, x_2) \leftarrow F^{-1}(c)$ from $\text{DECAP}(ct = (c, \sigma))$. However, hash query to $H(pk, x_1^*, x_2^*)$ never occurs in DECAP oracle in G_3 , since $\text{DECAP}(ct = (c, \sigma))$ returns $\perp$ if $c = c^*$ (which is equivalent to $(x_1, x_2) = (x_1^*, x_2^*)$ due to the injectivity of F). Therefore, $\mathcal{C}'$ only has to count j in $\mathcal{A}$'s hash queries to H after the challenge ciphertext generation, and hence (2) follows smoothly from Lemma 23.

Define **Ext** as the event that $(\hat{x}_1, \hat{x}_2) = (x_1^*, x_2^*)$. To bound $\Pr[\text{Ext in } G_3]$, we further hybrid G_3 from $G_{3.1}$ to $G_{3.8}$. We will show $|\Pr[\text{Ext in } G_{3.i}] - \Pr[\text{Ext in } G_{3.(i-1)}]| = \text{negl}(\lambda)$ and $\Pr[\text{Ext in } G_{3.8}] = \text{negl}(\lambda)$. Games $G_{3.1}$ - $G_{3.8}$ are shown in Fig. 9.

Game $G_{3.1}$. $G_{3.1}$ is the same as G_3 , except that $\mathcal{C}$ samples $j \leftarrow_{\$} [q'_H]$ and measures $\mathcal{A}$'s j -th post-challenge query to H to obtain $(\hat{x}_1, \hat{x}_2)$. It is easy to see that $\mathcal{C}$ in $G_{3.1}$ is just $\mathcal{C}'$ in G_3 . Therefore, $G_{3.1}$ is identical to G_3 . So (2) becomes

$$|\Pr[G_3 \Rightarrow 1] - \Pr[G_2 \Rightarrow 1]| \leq 2q'_H \cdot \sqrt{\Pr[\text{Ext in } G_{3.1}]} + \frac{6\sqrt{q_H - q'_H}}{2^{\kappa/4}}.$$

Game $G_{3.2}$. $G_{3.2}$ is the same as $G_{3.1}$, except that challenger $\mathcal{C}$ uses extractable random oracle simulator $\text{eRO} = (\text{eRO.RO}, \text{eRO.E})$ to simulate random oracle H . More precisely, $\mathcal{C}$ uses eRO.RO to simulate H both for the hash evaluations in the DECAP oracle and for $\mathcal{A}$'s hash query to H . $\mathcal{C}$ also main-

GAME SETUP $G_{3.1}-G_{3.8}$	$DEC_{ct^*}(ct)$ $G_{3.1}-G_{3.3}$
01: $(pk, sk) := (F, F^{-1})$	29: If $ct = ct^*$: Return $\perp$
02: $b \leftarrow_s \{0, 1\}$, Ext := 0, Find := 0	30: Parse $ct^* := (c^*, \sigma^*)$
03: $x_1^* \leftarrow \psi_1$, $i \leftarrow_s [q_G + q_d + 1]$ //G _{3.8}	31: Parse $ct := (c, \sigma)$
04: $x_2^* \leftarrow \psi_2$	32: If $c = c^*$: Return $\perp$
05: $(m_0, m_1, st) \leftarrow \mathcal{A}^{G, H, \text{Decap}^\perp}(pk)$ //G _{3.1}	33: $(x_1, x_2) := F^{-1}(c)$
06: $(m_0, m_1, st) \leftarrow \mathcal{A}^{G, \text{eRO.RO}, \text{Decap}^\perp}(pk)$ //G _{3.2} -G _{3.6}	34: $\mathcal{L} := \mathcal{L} \cup \{(x_1, x_2, \sigma)\}$ //G _{3.2} -G _{3.3}
//H is replaced with eRO.RO	35: $(pk', x'_1, x'_2) \leftarrow \text{eRO.E}(f, \sigma)$ //G _{3.3}
07: $(m_0, m_1, st) \leftarrow \mathcal{A}^{\bar{G} \setminus \{x_2^*\}, \text{eRO.RO}, \text{Decap}^\perp}(pk)$ //G _{3.7}	36: If $F(x_1, x_2) \neq c$: Return $\perp$
//G is replaced with $\bar{G} \setminus \{x_2^*\}$	37: $(\sigma', k) := H(pk, x_1, x_2)$ //G _{3.1}
08: $(m_0, m_1, st) \leftarrow \mathcal{A}^{\bar{G}, \text{eRO.RO}, \text{Decap}^\perp}(pk)$ //G _{3.8}	38: $(\sigma', k) := \text{eRO.RO}(pk, x_1, x_2)$ //G _{3.2} -G _{3.3}
09: $x_1^* := \text{Encode}(m_b, G(x_2^*))$ //G _{3.1} -G _{3.6}	39: If $\sigma' \neq \sigma$: Return $\perp$
10: $u \leftarrow_s \mathcal{U}$, $x_1^* := \text{Encode}(m_b, u)$ //G _{3.7}	40: $r := G(x_2)$
11: $c^* := F(x_1^*, x_2^*)$	41: $m := \text{Decode}(x_1, r)$
12: $\sigma^* \leftarrow_s \{0, 1\}^\lambda$, $k^* \leftarrow_s \{0, 1\}^\lambda$	42: Return (m, k)
13: $ct^* := (c^*, \sigma^*)$, $k_0^* := k^*$, $k_1^* \leftarrow_s \{0, 1\}^\lambda$	
14: $j \leftarrow_s [q_H]$	$DEC_{ct^*}(ct)$ $G_{3.4}-G_{3.7}$
15: $b' \leftarrow \mathcal{A}^{G, H, \text{Decap}_{ct^*}}(st, ct^*, k^*)$ //G _{3.1}	43: If $ct = ct^*$: Return $\perp$
16: $b' \leftarrow \mathcal{A}^{G, \text{eRO.RO}, \text{Decap}_{ct^*}}(st, ct^*, k^*)$ //G _{3.2} -G _{3.6}	44: Parse $ct^* := (c^*, \sigma^*)$
//H is replaced with eRO.RO	45: Parse $ct := (c, \sigma)$
17: $b' \leftarrow \mathcal{A}^{\bar{G} \setminus \{x_2^*\}, \text{eRO.RO}, \text{Decap}_{ct^*}}(st, ct^*, k^*)$ //G _{3.7}	46: If $c = c^*$: Return $\perp$
//G is replaced with $\bar{G} \setminus \{x_2^*\}$	47: $(x_1, x_2) := F^{-1}(c)$ //G _{3.4}
18: $b' \leftarrow \mathcal{A}^{\bar{G}, \text{eRO.RO}, \text{Decap}_{ct^*}}(st, ct^*, k^*)$ //G _{3.8}	48: $\mathcal{L} := \mathcal{L} \cup \{(x_1, x_2, \sigma)\}$ //G _{3.4}
19: Measure $\mathcal{A}$'s j -th query to H to obtain $(\hat{x}_1, \hat{x}_2)$	49: $(pk', x'_1, x'_2) \leftarrow \text{eRO.E}(f, \sigma)$
20: If $(\hat{x}_1, \hat{x}_2) = (x_1^*, x_2^*)$: Ext := 1	50: If $(pk, x_1, x_2) \neq (pk', x'_1, x'_2)$: //G _{3.4}
21: $\hat{\sigma} := G(\hat{x}_2)$ //G _{3.6}	Return $\perp$
22: $\hat{\sigma} := \bar{G} \setminus \{x_2^*\}(\hat{x}_2)$ //G _{3.7}	51: If $pk \neq pk'$: Return $\perp$ //G _{3.5} -G _{3.8}
23: If $\hat{x}_2 = x_2^*$: Find := 1 //G _{3.7}	52: If $F(x'_1, x'_2) \neq c$: Return $\perp$
24: $\hat{\sigma} := \bar{G}(\hat{x}_2)$ //G _{3.8}	53: $(\sigma', k) := \text{eRO.RO}(pk, x'_1, x'_2)$
25: For all $(x_1, x_2, \sigma) \in \mathcal{L}$:	54: If $\sigma' \neq \sigma$: Return $\perp$
$(pk', x'_1, x'_2) \leftarrow \text{eRO.E}(f, \sigma)$ //G _{3.2}	55: $r := G(x_2)$ //G _{3.4} -G _{3.6}
26: Define event Bad : $\exists (x_1, x_2, \sigma) \in \mathcal{L}$ s.t. //G _{3.2} -G _{3.4}	56: $r := \bar{G} \setminus \{x_2^*\}(x'_2)$ //G _{3.7}
$(pk, x_1, x_2) \neq (pk', x'_1, x'_2) \wedge \text{eRO.RO}(pk, x_1, x_2) = \sigma$	57: If $x'_2 = x_2^*$: Find := 1 //G _{3.7}
27: Measure $\mathcal{A}$'s i -th query to $\bar{G}$ to obtain $\hat{x}_2$ //G _{3.8}	58: $r := \bar{G}(x'_2)$ //G _{3.8}
28: Return Ext	59: $m := \text{Decode}(x'_1, r)$
	60: Return (m, k)

Figure 9: $G_{3.1}-G_{3.8}$ for the proof of Theorem 15. If a code line has no “//”, this line occurs in all games.

tains a list $\mathcal{L}$ to store each tuple (x_1, x_2, σ) , where $(x_1, x_2) := F^{-1}(c)$ is generated in oracle $\text{DECAP}(ct = (c, \sigma))$. At the end of the game, $\mathcal{C}$ extracts (pk', x'_1, x'_2) by invoking $(pk', x'_1, x'_2) \leftarrow \text{eRO.E}(f, \sigma)$ for each $(x_1, x_2, \sigma) \in \mathcal{L}$ where $f((pk, x_1, x_2), (\sigma, k)) := \sigma$. Since the extraction is done at the end of the game, by Theorem 19 (1), we have

$$\Pr[\text{Ext in } \mathbf{G}_{3.2}] = \Pr[\text{Ext in } \mathbf{G}_{3.1}].$$

Meanwhile, we define the following event **Bad** in $\mathbf{G}_{3.2}$:

$$\text{Bad} : \exists (x_1, x_2, \sigma) \in L \text{ s.t. } (pk, x_1, x_2) \neq (pk', x'_1, x'_2) \wedge \text{eRO.RO}(pk, x_1, x_2) = \sigma,$$

where $(pk', x'_1, x'_2) \leftarrow \text{eRO.E}(f, \sigma)$. By Theorem 19 (3), we know that $\Pr[\text{Bad in } \mathbf{G}_3] \leq \frac{300(q_H + q_d + 1)^3 \Gamma'(f) + 2}{2^{2\lambda}} + 8q_d^2 \sqrt{\frac{2\Gamma(f)}{2^{2\lambda}}}$. Note that $\Gamma(f) = \Gamma'(f) = 2^\lambda$ for $f(x, y_1 | y_2) = y_1$. Therefore, we have $\Pr[\text{Ext} \wedge \neg \text{Bad in } \mathbf{G}_{3.2}] \geq \Pr[\text{Ext in } \mathbf{G}_{3.2}] - \Pr[\text{Bad in } \mathbf{G}_{3.2}] \geq \Pr[\text{Ext in } \mathbf{G}_{3.2}] - \frac{300(q_H + q_d + 1)^3 + 2}{2^\lambda} - \frac{8\sqrt{2}q_d^2}{2^{\lambda/2}}$.

Game $\mathbf{G}_{3.3}$. In $\mathbf{G}_{3.3}$, for each $\text{DECAP}(ct = (c, \sigma))$, if $c \neq c^*$, then challenger $\mathcal{C}$ immediately extracts (pk', x'_1, x'_2) from $\text{eRO.E}(f, \sigma)$, instead of doing the extraction at the end of the game. By Theorem 19 (2.c), each swap of an invocation of eRO.RO with an invocation of eRO.E query affects the probability by at most $8\sqrt{2\Gamma(f)/2^{2\lambda}} = \frac{8\sqrt{2}}{2^{\lambda/2}}$. There are at most q_d queries to eRO.E and at most $(q_d + q_H)$ queries to eRO.RO . By a union bound we have

$$\Pr[\text{Ext} \wedge \neg \text{Bad in } \mathbf{G}_{3.3}] \geq \Pr[\text{Ext} \wedge \neg \text{Bad in } \mathbf{G}_{3.2}] - \frac{8\sqrt{2}q_d(q_d + q_H)}{2^{\lambda/2}}.$$

Game $\mathbf{G}_{3.4}$. $\mathbf{G}_{3.4}$ is the same as $\mathbf{G}_{3.3}$, except for the simulation of DECAP oracle.

In $\mathbf{G}_{3.4}$, upon a decryption query $\text{DECAP}(ct = (c, \sigma))$ query from $\mathcal{A}$, if $c \neq c^*$, challenger $\mathcal{C}$ computes $(x_1, x_2) := F^{-1}(c)$ and $(pk', x'_1, x'_2) \leftarrow \text{eRO.E}(f, \sigma)$, just like $\mathbf{G}_{3.3}$. Next, a new rejection rule is added in $\mathbf{G}_{3.4}$.

• **Rejection rule I:** If $(pk, x_1, x_2) \neq (pk', x'_1, x'_2)$, then $\mathcal{C}$ returns $\perp$ directly.

Otherwise, $\mathcal{C}$ computes $\sigma' := \text{eRO.RO}(pk', x'_1, x'_2)$, checks whether $c = F(x'_1, x'_2)$, and then proceeds with (x'_1, x'_2) just like $\mathcal{C}$ does in $\mathbf{G}_{3.3}$ with (x_1, x_2) .

Note that $\mathbf{G}_{3.4}$ and $\mathbf{G}_{3.3}$ differs only when $\mathcal{A}$ makes a decryption query $\text{DECAP}(ct = (c, \sigma))$ s.t. $\mathcal{C}$ returns $m \neq \perp$ in $\mathbf{G}_{3.3}$ but returns $\perp$ due to the rejection rule in $\mathbf{G}_{3.4}$. This implies there exists $(pk', x'_1, x'_2) \neq (pk, x_1, x_2) \wedge \text{eRO.RO}(pk, x_1, x_2) = \sigma$ for a decryption query $\text{DECAP}(ct = (c, \sigma))$, which means event **Bad** happens in $\mathbf{G}_{3.3}$. In other words, if event **Bad** does not happen in $\mathbf{G}_{3.3}$, $\mathbf{G}_{3.4}$ and $\mathbf{G}_{3.3}$ are the same. So we have $\Pr[\text{Ext} \wedge \neg \text{Bad in } \mathbf{G}_{3.4}] = \Pr[\text{Ext} \wedge \neg \text{Bad in } \mathbf{G}_{3.3}]$. Obviously, $\Pr[\text{Ext in } \mathbf{G}_{3.4}] \geq \Pr[\text{Ext} \wedge \neg \text{Bad in } \mathbf{G}_{3.4}]$.

Game $\mathbf{G}_{3.5}$. $\mathbf{G}_{3.5}$ is the same as $\mathbf{G}_{3.4}$, except that we replace the Rejection rule I with the following Rejection rule II.

• **Rejection rule II:** If $pk \neq pk'$, then $\mathcal{C}$ returns $\perp$ directly.

Otherwise, $\mathcal{C}$ computes $\sigma' := \text{eRO.RO}(pk', x'_1, x'_2)$, checks whether $c = F(x'_1, x'_2)$, and then proceeds just like $\mathcal{C}$ does in $\mathbf{G}_{3.4}$ with (x'_1, x'_2) .

Note that $\mathsf{G}_{3.5}$ and $\mathsf{G}_{3.4}$ differs only in the case that $\mathcal{A}$ makes a decryption query $\text{DECAP}(ct = (c, \sigma))$ s.t. $\mathcal{C}$ returns $m \neq \perp$ in $\mathsf{G}_{3.5}$ but returns $\perp$ due to Rejection rule I in $\mathsf{G}_{3.4}$. Let $(x_1, x_2) := \mathsf{F}^{-1}(c)$, $(pk', x'_1, x'_2) \leftarrow \text{eRO.E}(f, \sigma)$. On the one hand, $\mathcal{C}$ returns $m \neq \perp$ in $\mathsf{G}_{3.5}$ implies that Rejection rule II does not hold, so $pk = pk'$. On the other hand, $\mathcal{C}$ returns $\perp$ due to the Rejection rule I in $\mathsf{G}_{3.4}$ means $(pk, x_1, x_2) \neq (pk', x'_1, x'_2)$. Therefore, it must hold that $(x_1, x_2) \neq (x'_1, x'_2)$. We also know that $\mathsf{F}(x'_1, x'_2) = c$ since $\text{DECAP}(c, \sigma) \neq \perp$ in $\mathsf{G}_{3.5}$. Then we have $(x_1, x_2) := \mathsf{F}^{-1}(c) = \mathsf{F}^{-1}(\mathsf{F}(x'_1, x'_2))$, which is contradictory to the injectivity of the underlying F . Therefore, the above case does not happen, and $\mathsf{G}_{3.5}$ is the same as $\mathsf{G}_{3.4}$. So we have $\Pr[\text{Ext in } \mathsf{G}_{3.5}] = \Pr[\text{Ext in } \mathsf{G}_{3.4}]$.

Thanks to eRO.E and Rejection rule II, $\mathcal{C}$ can answer DECAP queries without F^{-1} from $\mathsf{G}_{3.5}$ on.

Game $\mathsf{G}_{3.6}$. In $\mathsf{G}_{3.6}$, after $\mathcal{C}$ obtains $(\hat{x}_1, \hat{x}_2)$ by measuring $\mathcal{A}$'s j -th post-challenge hash query to H , $\mathcal{C}$ will make an additional query to $G(\hat{x}_2)$. This query $G(\hat{x}_2)$ does not affect the probability of Ext . So we have $\Pr[\text{Ext in } \mathsf{G}_{3.6}] = \Pr[\text{Ext in } \mathsf{G}_{3.5}]$. This additional query is used to remove the square root loss by applying Semi-O2H lemma in the following game.

Game $\mathsf{G}_{3.7}$. In $\mathsf{G}_{3.7}$, $\mathcal{C}$ introduces a new random oracle $\bar{G}$ such that $\bar{G}(x_2) = G(x_2)$ for all $x_2 \neq x_2^*$ and $\bar{G}(x_2^*) \leftarrow_s \mathcal{U}$. $\mathcal{C}$ still computes $x_1^* := \text{Encode}(m_b, G(x_2^*))$ with random oracle G . However, $\mathcal{C}$ uses the punctured oracle $\bar{G} \setminus \{x_2^*\}$ (cf. Definition 20) to respond any (quantum) random oracle query to G (both from $\mathcal{A}$'s queries and from oracle DECAP). Now that $\mathcal{A}$ has no information of $G(x_2^*)$, so $x_1^* := \text{Encode}(m_b, G(x_2^*))$ has the same distribution as $x_1^* := \text{Encode}(m_b, u)$ for $u \leftarrow_s \mathcal{U}$. By the semi-classical O2H lemma (Lemma 21), we have

$$\sqrt{\Pr[\text{Ext in } \mathsf{G}_{3.6}]} \leq \sqrt{\Pr[\text{Ext} \wedge \neg \text{Find in } \mathsf{G}_{3.7}]} + 2\sqrt{(q_G + q_d + 2)\Pr[\text{Find in } \mathsf{G}_{3.7}]}.$$

Here Find is defined as follows. For each $\mathcal{C}$'s (quantum) random oracle query $|x\rangle$ to $G \setminus \{x_2^*\}$, it first computes $|x\rangle|f_{x_2^*}(x)\rangle$ with an auxiliary qubit, where $f_{x_2^*}(x) = 1$ if $x_2^* = x_2$ and $f_{x_2^*}(x) = 0$ otherwise. Then it measures the auxiliary qubit $|f_{x_2^*}(x)\rangle$, and forwards the (collapsed) qubit $|x\rangle$ to hash function G . Find happens if any of the measurements of $|f_{x_2^*}(x)\rangle$ outputs 1.

Note that $\Pr[\text{Ext} \wedge \neg \text{Find in } \mathsf{G}_{3.7}] = 0$. The reason is as follows. Recall that $\mathcal{C}$ will query $\bar{G} \setminus \{x_2^*\}(\hat{x}_2)$ after it obtains $(\hat{x}_1, \hat{x}_2)$ by measuring $\mathcal{A}$'s j -th query to H . If Ext occurs, which means $(\hat{x}_1, \hat{x}_2) = (x_1^*, x_2^*)$, then this hash query $\bar{G} \setminus \{x_2^*\}(\hat{x}_2)$ must trigger the event Find . So we have $\Pr[\text{Ext} \wedge \neg \text{Find in } \mathsf{G}_{3.7}] = 0$ and thus

$$\Pr[\text{Ext in } \mathsf{G}_{3.6}] \leq 4(q_G + q_d + 2)\Pr[\text{Find in } \mathsf{G}_{3.7}].$$

Game $\mathsf{G}_{3.8}$. $\mathsf{G}_{3.8}$ is the same as $\mathsf{G}_{3.7}$ except for two changes. The first change in $\mathsf{G}_{3.8}$ is the generation of x_1^* . In $\mathsf{G}_{3.8}$, $\mathcal{C}$ samples $x_1^* \leftarrow \psi_1$ at the beginning of the game. Recall that in $\mathsf{G}_{3.7}$, $x_1^* := \text{Encode}(m_b, u)$ for $u \leftarrow_s \mathcal{U}$. By the ψ_1 -message hiding property of underlying SOTP, we know that $x_1^* = \text{Encode}(m_b, u)$ follows distribution ψ_1 . Therefore, this is only a conceptual change.

The second change is that $\mathcal{C}$ uses $\bar{G}$ rather than the punctured oracle $\bar{G} \setminus \{x_2^*\}$ to respond to any (quantum) random oracle query to G . Besides, $\mathcal{C}$ randomly

samples $i \leftarrow_{\$} [q_G + q_d + 1]$, and measures the i -th query to G to obtain $\hat{x}_2$. By Lemma 22, we have $\Pr[\text{Find in } \mathsf{G}_{3.7}] \leq 4(q_G + q_d + 2) \cdot \Pr[\hat{x}_2 = x_2^* \text{ in } \mathsf{G}_{3.8}]$.

Now in $\mathsf{G}_{3.8}$, $\mathcal{C}$ can simulate the whole game only with F and c^* . So we are ready to prove $\Pr[\hat{x}_2 = x_2^* \text{ in } \mathsf{G}_{3.8}] \leq \text{Adv}_{\mathsf{F}, \mathsf{F}^{-1}, \psi}^{\text{PD-OW}}(\mathcal{B})$ with a security reduction.

Let $\mathcal{B}$ be an algorithm that takes as input (F, c^*) with $c^* := \mathsf{F}(x_1^*, x_2^*)$ where $(x_1^*, x_2^*) \leftarrow (\psi_1, \psi_2)$, and aims to find x_2^* . To this end, $\mathcal{B}$ first samples $i \leftarrow_{\$} [q_G + q_d + 1]$ and simulates $\mathsf{G}_{3.8}$ with (F, c^*) and random oracle $\bar{G}$ just before the i -th query to $\bar{G}$. Then $\mathcal{B}$ measures the argument of the i -th query in the computational basis and outputs $\hat{x}_2$. Clearly, up to now $\mathcal{B}$ perfectly simulates $\mathsf{G}_{3.8}$ for $\mathcal{A}$. If $\hat{x}_2 = x_2^*$ happens in $\mathsf{G}_{3.8}$, then $\mathcal{B}$ wins. Therefore, we have

$$\Pr[\hat{x}_2 = x_2^* \text{ in } \mathsf{G}_{3.8}] \leq \text{Adv}_{\mathsf{F}, \mathsf{F}^{-1}, \psi}^{\text{PD-OW}}(\mathcal{B}).$$

Backtracking all the above (in)equalities from $\mathsf{G}_{3.8}$ to G_3 and set $q := q_G + q_H + q_d + 2$, we obtain that

$$|\Pr[\mathsf{G}_3 \Rightarrow 1] - \Pr[\mathsf{G}_2 \Rightarrow 1]| \leq 2q \sqrt{16q^2 \text{Adv}_{\mathsf{F}, \mathsf{F}^{-1}, \psi}^{\text{PD-OW}}(\mathcal{B}) + \frac{16\sqrt{2}q^2}{2^{\lambda/2}} + \frac{300q^3}{2^\lambda} + \frac{6\sqrt{q}}{2^{\kappa/4}}}.$$

Game G_4 . G_4 is the same as G_3 , except that for two changes. Firstly, $\mathcal{C}$ uses the extractable random oracle simulator eRO to simulate random oracle H , and simulate DECAP without usage of F^{-1} . Secondly, upon a decryption query $\text{DECAP}(ct = (c, \sigma))$, if $pk \neq pk'$, then $\mathcal{C}$ returns $\perp$ directly, where $(pk', x_1, x_2) := \text{eRO.E}(f, \sigma)$. With the same argument from $\mathsf{G}_{3.2}$ to $\mathsf{G}_{3.5}$, we have

$$|\Pr[\mathsf{G}_4 \Rightarrow 1] - \Pr[\mathsf{G}_3 \Rightarrow 1]| \leq \frac{16\sqrt{2}q^2}{2^{\lambda/2}} + \frac{300q^3}{2^\lambda}.$$

Now $\mathcal{C}$ can simulate the recovery of (x_1, x_2) in oracle DECAP without F^{-1} .

Game G_5 . In G_5 , $\mathcal{C}$ introduces a new random oracle $\bar{G}$ satisfying $\bar{G}(x_2) = G(x_2)$ for all $x_2 \neq x_2^*$ and $\bar{G}(x_2^*) \leftarrow_{\$} \mathcal{U}$. $\mathcal{C}$ still computes $x_1^* := \text{Encode}(m_b, G(x_2^*))$ with random oracle G . However, $\mathcal{C}$ uses the new oracle $\bar{G}$ to respond to (quantum) random oracle queries to G from $\mathcal{A}$ and the DECAP oracle. By the semi-classical O2H lemma (Lemma 21), we have

$$|\Pr[\mathsf{G}_5 \Rightarrow 1] - \Pr[\mathsf{G}_4 \Rightarrow 1]| \leq 2\sqrt{(q_G + 1)\Pr[\text{Find in } \mathsf{G}_5^{\bar{G} \setminus \{x_2^*\}}]},$$

where $\mathsf{G}_5^{\bar{G} \setminus \{x_2^*\}}$ is similar to G_5 except that $\bar{G}$ is replaced by $\bar{G} \setminus \{x_2^*\}$. Find is the event that any of the measurement of $|f_{x_2^*}(x)\rangle$ returns 1 in $\mathsf{G}_5^{\bar{G} \setminus \{x_2^*\}}$.

Now in G_5 , $\mathcal{A}$ has no information of $G(x_2^*)$, so $G(x_2^*)$ is uniform. By the message hiding property of underlying SOTP, $\text{Encode}(m_0; G(x_2^*))$ must have the same distribution as that of $\text{Encode}(m_1; G(x_2^*))$. So we have $\Pr[\mathsf{G}_5 \Rightarrow 1] = 1/2$.

The remaining is to bound the probability of the event Find in $\mathsf{G}_5^{\bar{G} \setminus \{x_2^*\}}$.

Game G_6 . G_6 is similar to G_5 , except that $\mathcal{C}$ first samples $i \leftarrow_{\$} [q_G]$ and measures the argument of the i -th query to $\bar{G}$ in the computational basis and outputs

x'_2 . If $x'_2 = x_2^*$, then Find happens in G_6 . By Lemma 22, we have

$$\Pr \left[\text{Find in } G_5^{\overline{G} \setminus \{x_2^*\}} \right] \leq 4q_G \cdot \Pr [\text{Find in } G_6].$$

Now we prove $\Pr [\text{Find in } G_6] \leq \text{Adv}_{F, F^{-1}, \psi}^{\text{PD-OW}}(\mathcal{B})$. Let $\mathcal{B}$ be an algorithm that takes as input (F, c^*) with $c^* := F(x_1^*, x_2^*)$ where $(x_1^*, x_2^*) \leftarrow (\psi_1, \psi_2)$. $\mathcal{B}$'s aim is to find x_2^* . To this end, $\mathcal{B}$ first samples $i \leftarrow_{\$} [q_G]$ and runs $\mathcal{C}$ in G_6 with (F, c^*) and random oracle $\overline{G}$ just before $\mathcal{C}$ makes the i -th query to $\overline{G}$. Then $\mathcal{B}$ measures the argument of the query in the computational basis and outputs x'_2 . Note that $\mathcal{B}$ perfectly simulates G_6 . If Find happens, then $x'_2 = x_2^*$ and $\mathcal{B}$ wins. So we have

$$\Pr [\text{Find in } G_6] \leq \text{Adv}_{F, F^{-1}, \psi}^{\text{PD-OW}}(\mathcal{B}).$$

Backtracking all the above (in)equalities and set $q := q_G + q_H + q_d + 2$, we get the following bound for any (F, F^{-1}) with injectivity:

$$\text{Adv}_{F, F^{-1}}^{\text{CCA}}(\mathcal{A}) \leq 16q^2 \sqrt{\text{Adv}_{F, F^{-1}, \psi}^{\text{PD-OW}}(\mathcal{B})} + 4q \sqrt{\frac{300q^3}{2^\lambda} + \frac{16\sqrt{2}q^2}{2^{\lambda/2}} + \frac{6\sqrt{q}}{2^\kappa/4}}.$$

For (F, F^{-1}) without injectivity, we have $\text{Adv}_{F, F^{-1}}^{\text{CCA}}(\mathcal{A}) \leq 1$. Taking the expectation over $(F, F^{-1}) \leftarrow \text{Gen}$ and applying Jensen's inequality, Theorem 15 follows. $\square$

A.3 Proofs of Additional Properties

We give the proof of PR-CCA security of our OAEP[†] transform. This implies the anonymity of our OAEP[†].

Proof of Theorem 16. The proof outline is almost the same as Theorem 15.

Game G'_0 . G'_0 is the original $\text{Exp}_{\text{KEM}}^{\text{PR-CCA}}$. So $\text{Adv}_{\text{KEM}}^{\text{PR-CCA}}(\mathcal{A}) = |\Pr [G'_0 \Rightarrow 1] - 1/2|$.

Game G'_1 . The hybrid from G'_0 to G'_1 is similar to the hybrid from G_0 to G_5 in the proof of Theorem 15, so we have

$$|\Pr [G'_1 \Rightarrow 1] - \Pr [G'_0 \Rightarrow 1]| \leq 16q^2 \sqrt{\text{Adv}_{F, \psi}^{\text{PD-OW}}(\mathcal{B})} + 4q \sqrt{\frac{300q^3}{2^\lambda} + \frac{8\sqrt{2}q^2}{2^{\lambda/2}} + \frac{6\sqrt{q}}{2^{\kappa/4}}} + \eta.$$

In G'_1 , the challenger generates the challenge ciphertext $ct_0^* := (c_0^*, \sigma_0^*)$ and the challenge key k_b^* by $c_0^* := F(x_1^*, x_2^*)$, $\sigma_0^* \leftarrow_{\$} \{0, 1\}^\lambda$ and $k_b^* \leftarrow_{\$} \{0, 1\}^\lambda$, where $(x_1^*, x_2^*) \leftarrow (\psi_1, \psi_2)$. The challenger simulates H with eRO.RO and simulates G with $\overline{G}$. It simulates DECAP oracle with eRO.E instead of F^{-1} . The description of G'_1 is shown in Fig. 10.

Next we show that $|\Pr [G'_1 \Rightarrow 1] - \frac{1}{2}| \leq \text{Adv}_{F, \psi}^{\text{PR}}(\mathcal{B})$. Let $\mathcal{B}$ be an algorithm that takes as input (F, c_b) , where $c_0 = F(x_1, x_2)$ when $b = 0$ and $c_1 \leftarrow_{\$} \mathcal{Y}$ when $b = 1$. $\mathcal{B}$ aims to guess the bit b . To this end, it simulates G'_1 for $\mathcal{A}$ as follows. $\mathcal{B}$ forwards $pk := F$ to $\mathcal{A}$ and simulates DECAP oracle with eRO. After $\mathcal{A}$ outputs (m, st) , $\mathcal{B}$ randomly samples $\sigma \leftarrow_{\$} \{0, 1\}^\lambda$ and $k_b^* \leftarrow_{\$} \{0, 1\}^\lambda$, outputs

GAME G'_1	$\text{DEC}_{ct_b^*}(ct)$
09: $(pk, sk) := (F, F^{-1})$	21: If $ct = ct_b^*$: Return $\perp$
10: $b \leftarrow_s \{0, 1\}$	22: Parse $ct_b^* := (c_b^*, \sigma_b^*)$
11: $x_1^* \leftarrow \psi_1, x_2^* \leftarrow \psi_2$	23: Parse $ct := (c, \sigma)$
12: $(m, st) \leftarrow \mathcal{A}^{\overline{G}, \text{eRO}, \text{RO}, \text{DECAP}_\perp}(pk)$	24: If $c = c_b^*$: Return $\perp$
13: $c_0^* := F(x_1^*, x_2^*)$	25: $(pk', x'_1, x'_2) := \text{eRO.E}(f, c)$
14: $\sigma_0^* \leftarrow_s \{0, 1\}^\lambda$	26: If $pk' \neq pk$: Return $\perp$
15: $ct_0^* := (c_0^*, \sigma_0^*), k_0^* \leftarrow_s \{0, 1\}^\lambda$	27: If $F(x'_1, x'_2) \neq c$: Return $\perp$
16: $c_1^* \leftarrow_s \mathcal{Y}$	28: $(\sigma', k) := \text{eRO.RD}(pk', x'_1, x'_2)$
17: $\sigma_1^* \leftarrow_s \{0, 1\}^\lambda$	29: If $\sigma' \neq \sigma$: Return $\perp$
18: $ct_1^* := (c_1^*, \sigma_1^*), k_1^* \leftarrow_s \{0, 1\}^\lambda$	30: $r := \overline{G}(x'_2)$
19: $b' \leftarrow \mathcal{A}^{\overline{G}, \text{eRO}, \text{RO}, \text{DECAP}_{ct_b^*}}(st, pk, ct_b^*, k_b^*)$	31: $m := \text{Decode}(x'_1, r)$
20: Return $b = b'$	32: Return (m, k)

Figure 10: G'_1 for proof in Theorem 16.

$ct_b^* = (c_b, \sigma)$ and k_b^* to $\mathcal{A}$. Finally, $\mathcal{B}$ just takes the guessing bit b' of $\mathcal{A}$ as its own output. Note that $\mathcal{B}$'s simulation of G'_1 is perfect for $\mathcal{A}$. Therefore, if $\mathcal{A}$ wins G'_1 , i.e., $G'_1 \Rightarrow 1$, then $\mathcal{B}$ breaks the pseudo-randomness of the ATIF family F . So $|\Pr[G'_1 \Rightarrow 1] - \frac{1}{2}| \leq \text{Adv}_{\mathcal{F}, \psi}^{\text{PR}}(\mathcal{B})$. This completes the proof of Theorem 16. $\square$

Now we show the proof of robustness of our OAEP[†] transform.

Proof of Theorem 17. Let $(pk_0, sk_0) = (F_0, F_0^{-1}) \leftarrow \text{Gen}$ and $(pk_1, sk_1) = (F_1, F_1^{-1}) \leftarrow \text{Gen}$. Then by the collision resistance of ATIF, we know that $pk_0 \neq pk_1$ overwhelmingly. Suppose $\mathcal{A}$ outputs a ciphertext $ct = (c, \sigma)$ and breaks the ROB-CCA security of KEM. Let $(x_1, x_2) := F_0^{-1}(c)$ and $(x'_1, x'_2) := F_1^{-1}(c)$. If $(\sigma, k) := H(pk_0, x_1, x_2)$ and $(\sigma', k') := H(pk_1, x'_1, x'_2)$, then we have $\sigma = \sigma'$. Further define $\tilde{H}(\cdot, \cdot, \cdot)$ as the first half output of H , then $\sigma = \tilde{H}(pk_0, x_1, x_2) = \tilde{H}(pk_1, x'_1, x'_2)$. This means that $\mathcal{A}$ has broken the collision resistance of the hash function $\tilde{H}$, which happens with probability at most $\frac{648(q_H+1)^3}{2^\lambda}$ by Lemma 24. $\square$